

How Neural Networks Learn

Dr. Benoit Mialet

benoit.mialet@hotmail.fr

benoit.mialet@edu.dsti.institute



School of Engineering

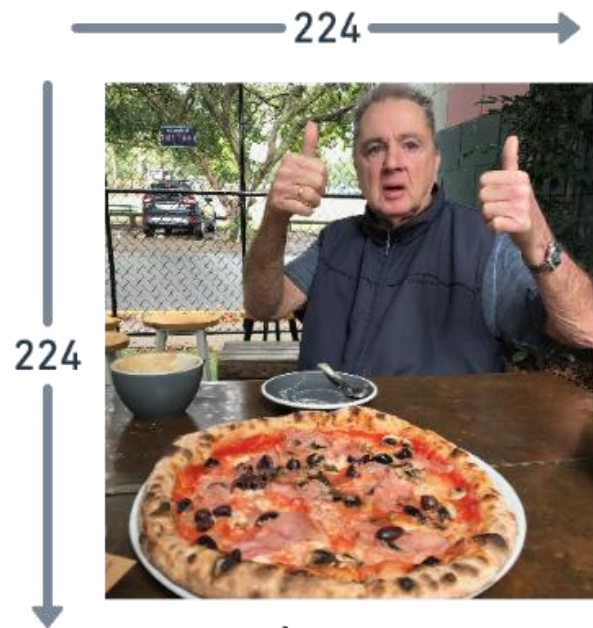
Data preprocessing

Why preprocess data?

- ❑ We have mentioned previously that input features of a Neural Network **have to be numerical** (integers, floats) so that the model can process them
- ❑ However, real-world data often includes heterogeneous types: text, categories, or images. These types of data cannot be used directly by a model
- ❑ The goal of data preprocessing is to:
 - transform raw data into a format that a neural network can learn
 - Apply scaling methods to make training more stable and efficient

Models only “eat” numerical data

- ❑ Some types of data, like text, categories, or images, cannot be processed directly by a Neural Network. They must first be converted into **numerical representations**
 - E.g. for an image, every pixel takes a numerical value
 - E.g. for text, every word, or piece of word, is converted into a vector
 - E.g. Categorical features must be converted into numbers.
- ❑ Words or categories conversion process is called **encoding**



[[[-0.01157, 0.02485, 0.02878...
-0.01271, 0.03971, 0.08827...
0.02680, 0.05589, -0.01068...
-0.00597, 0.00639, -0.01819...
...]]]

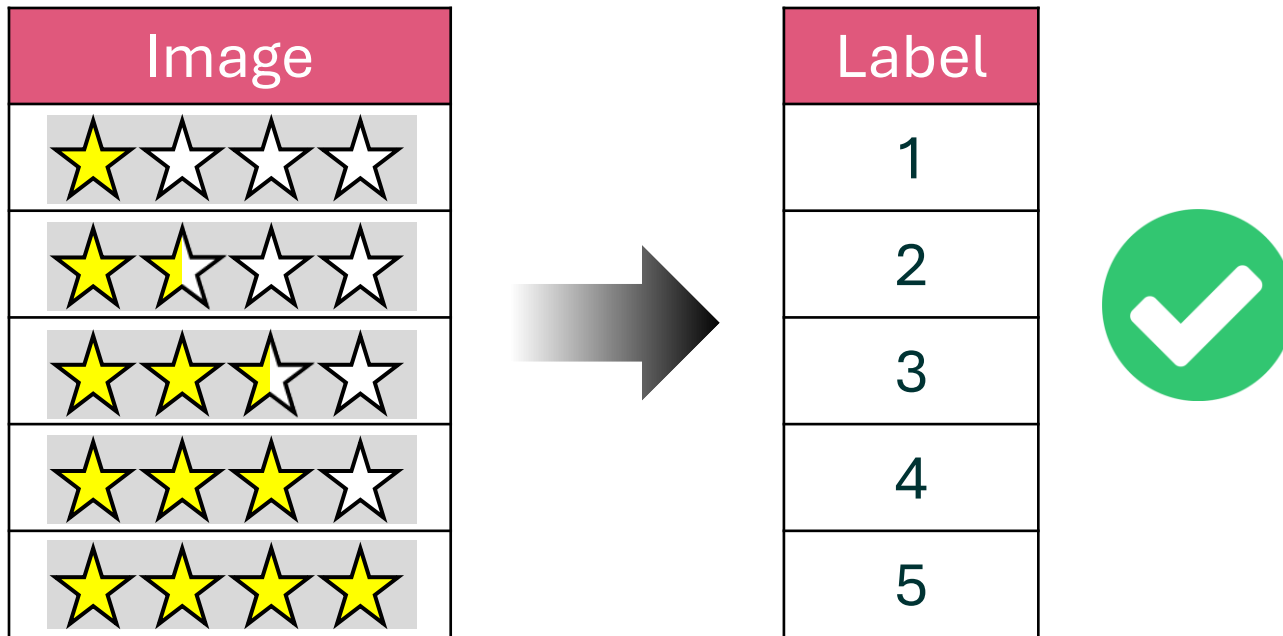


numerical representation of an image

Categorical Data Encoding

1 Ordinal Encoding converts each category into an integer value

- It works well **when categories have a natural order** (ordinal data)
- e.g. “*low*”, “*medium*”, “*high*”

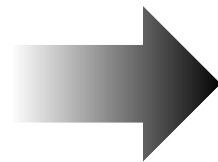


1 Ordinal Encoding converts each category into an integer value

- However, it is not suitable for most nominal (unordered categorical) features
- It implies a false numerical order for non-ordinal categories



Colour
Red
Green
Blue
Yellow
Gingerline



Colour
1
2
3
4
5

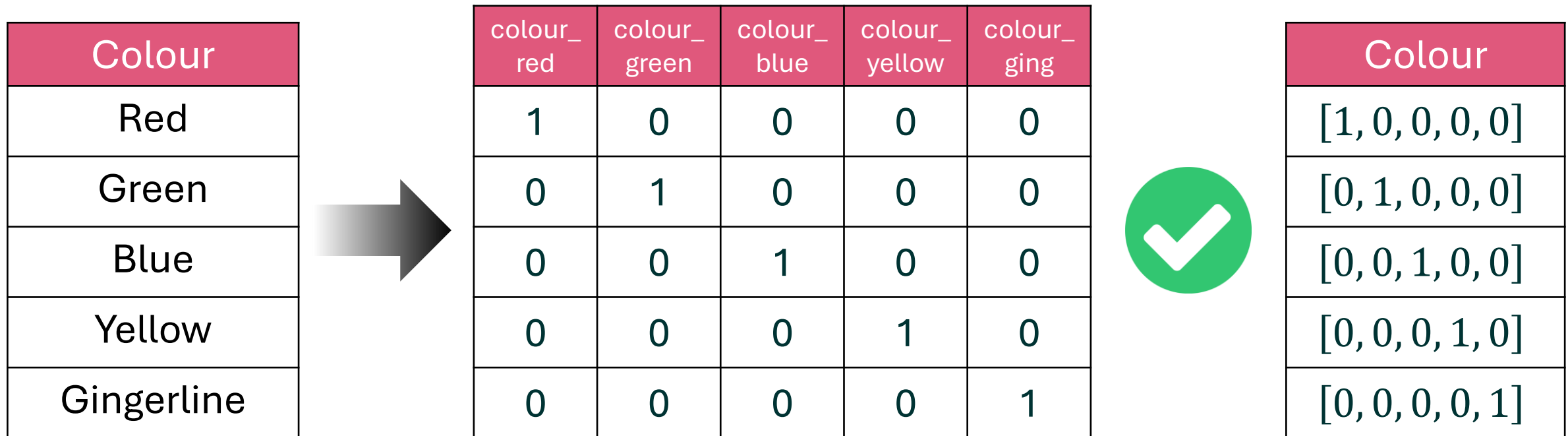


Model may assume
“Blue > Green”,
which is meaningless,
because there is no
hierarchical relation
between the colors

Categorical Data Encoding

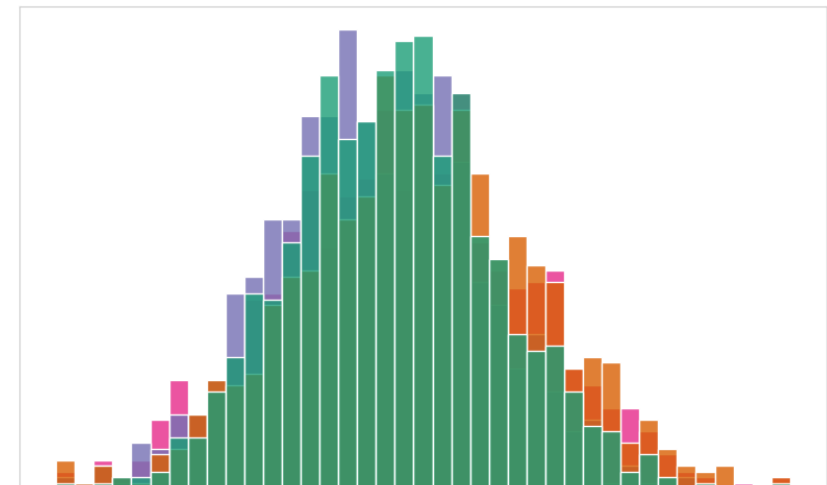
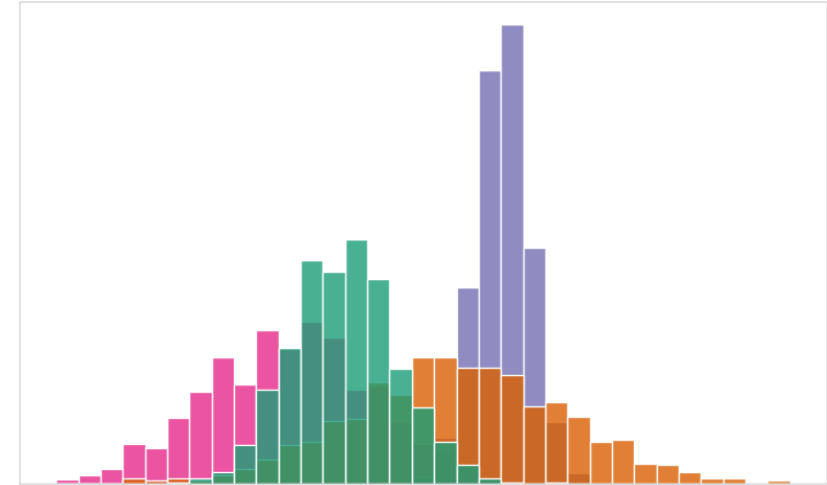
2 One-Hot Encoding represents each category as a binary vector, based on the number of categories, with only one bit = 1 per category

- It works well **when categories have no natural order** (nominal data)
- However, if the number of categories is large, one-hot encoding can create very high-dimensional vectors, leading to significant memory usage.



Data Normalisation

- ❑ Once all data is numerically represented, their format can be processed by a NN
- ❑ However, values can differ greatly in scale
 - Large values (e.g. income) dominate smaller ones (e.g. age)
- ❑ A Neural Network has no knowledge about features' meaning, and will establish relations as if they had the same unit. It is thus necessary to put all input features on a similar scale. This is **Feature Normalisation** or **Feature Scaling**
- ❑ ...We'll come back later to understand why scaling affects model training stability



1 Min-Max Scaling

- ❑ Subtract minimum value then divide by the data range
- ❑ Very sensitive to outliers (compresses normal data)
- ❑ Best for
 - Algorithms that require input features to be on a **bounded scale**
- ❑ Example use case: Image pixel values (0-255) scaled to 0-1 for neural network training

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} (u - l) + l$$

u, l , are respectively the upper and lower values of the new range for normalized data

X : any distribution



$0 \leq X' \leq 1$
(or any range you want)

2 Standardisation (Z-Score normalisation)

- ❑ Subtract the mean then divide by standard deviation
- ❑ Sensitive to outliers
- ❑ Best for
 - Algorithms that assume normally distributed data (linear regression, logistic regression, SVM, PCA)
 - **Neural networks in general** (inputs are close to 0), to be seen later
- ❑ Example usage: Scaling features like age and income

$$x' = \frac{x - \mu}{\sigma}$$

X: Normal distribution



X': Normal distribution

Mean = 0

Standard deviation = 1

The training cycle

1

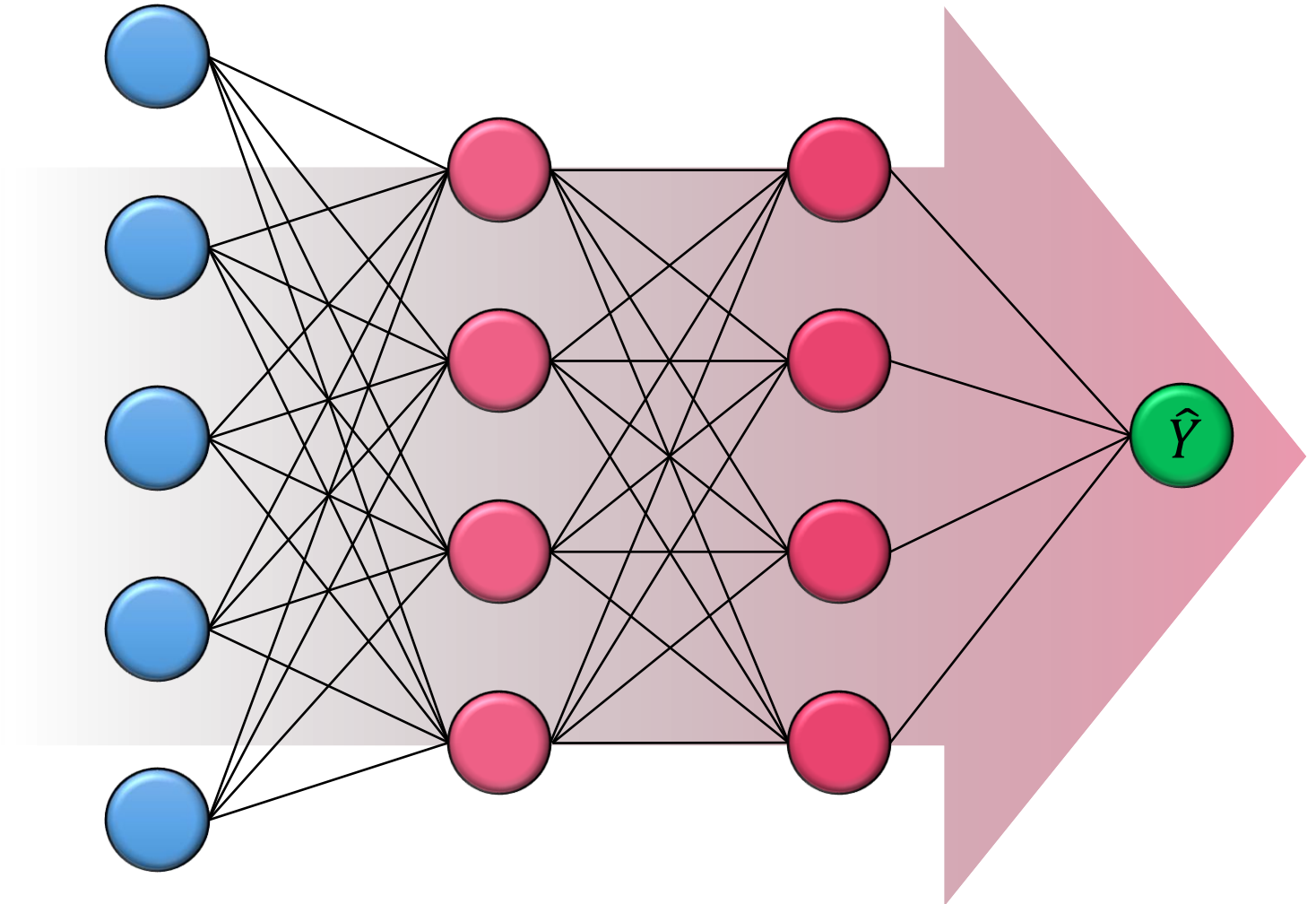
Data Loading

- ❑ Before training, data is first loaded in memory, on the device that will be used for machine learning (GPU(s) VRAM or CPU(s) RAM)
- ❑ Then, model is “**fed**” with data in 3 possible ways, depending on system resources
 - All samples at once
 - By batches of X samples
 - Samples one by one

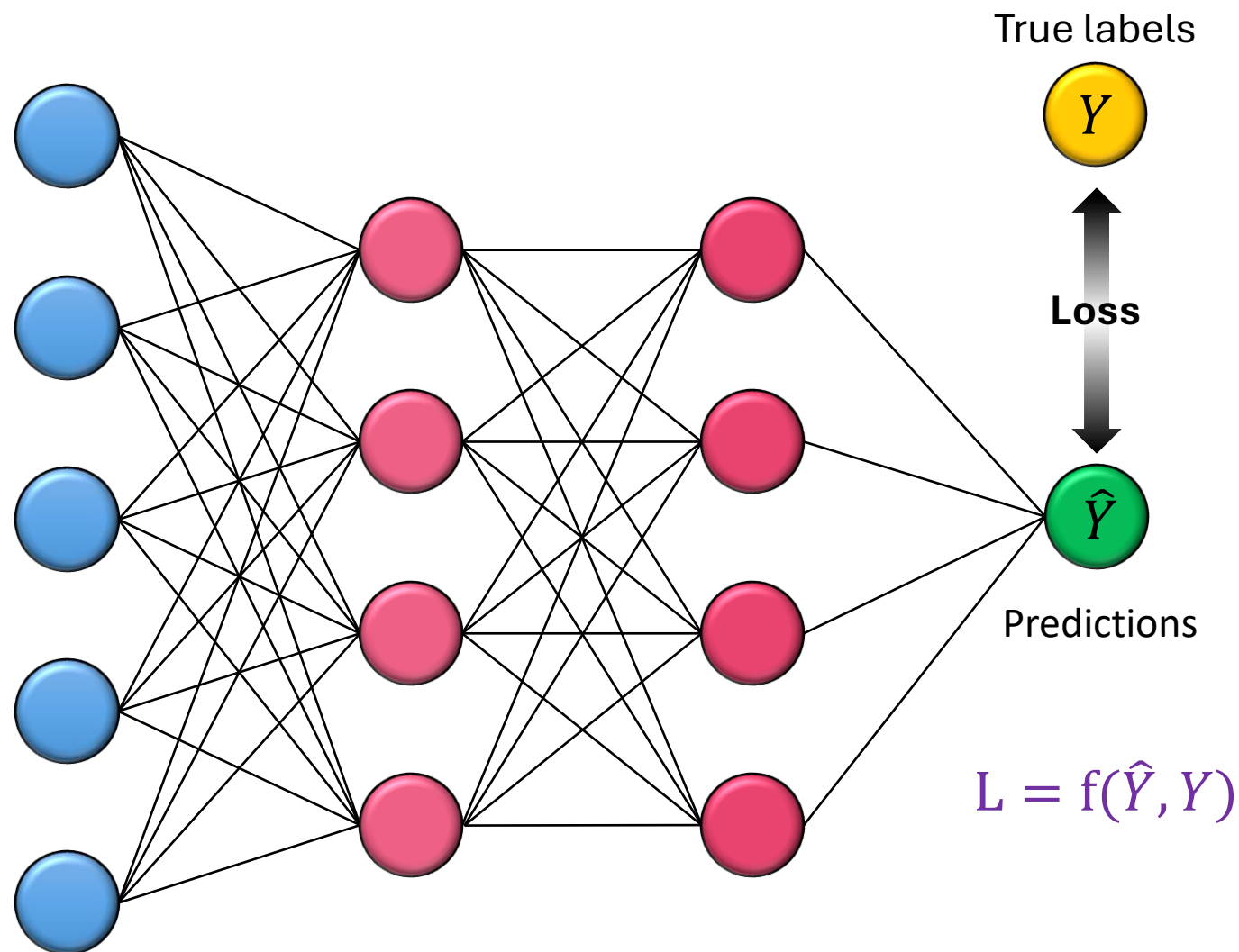
Loading...



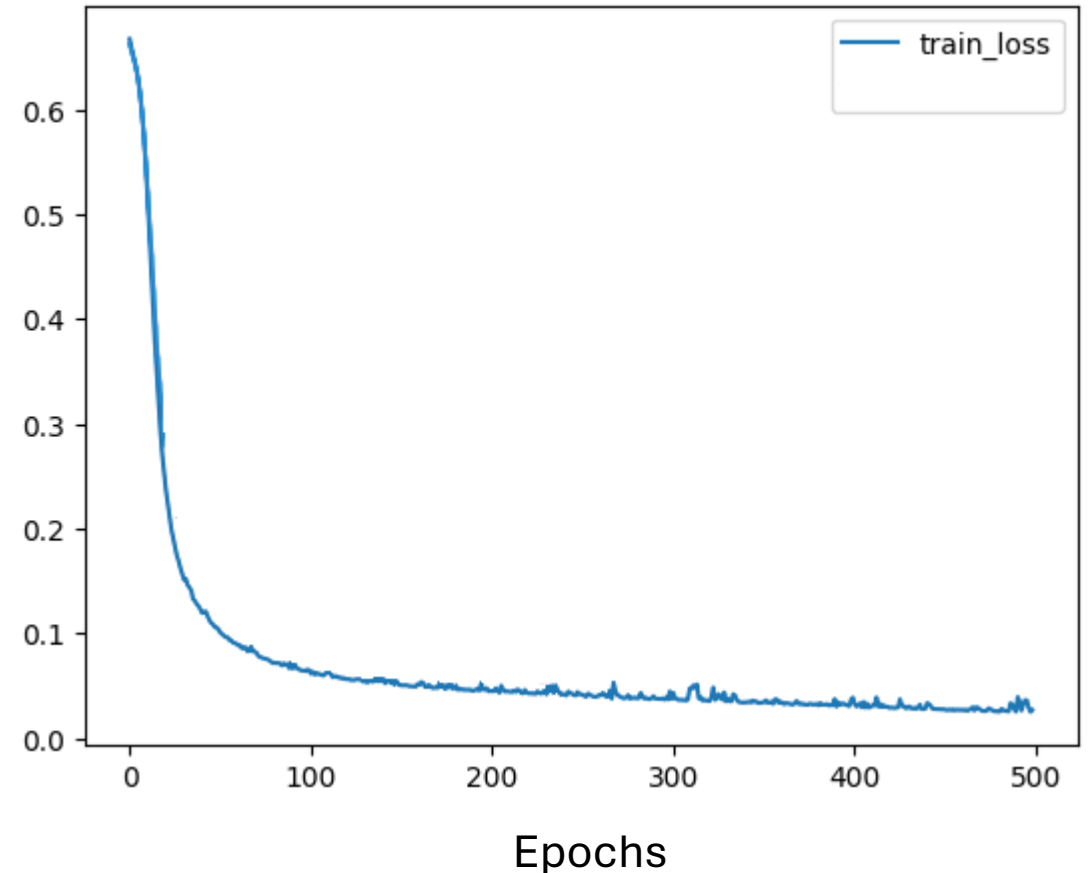
- ❑ input data is then passed forward through the network
- ❑ Each layer applies:
 - Linear transformation
$$z = \sum x_i \cdot w_i + b$$
 - Non-linear activation
$$a = \sigma(z)$$
- ❑ The output (logits or probabilities) is the model's prediction ($\hat{Y}$)



- ❑ **Loss Function** or **Cost Function** or **Objective function** measures how far the predictions ($\hat{Y}$) are from the true labels (Y)
- ❑ It gives a numerical value representing the model's **error** (or **Loss**, or **Cost**)
- ❑ Small loss means good predictions, high loss means poor predictions
- ❑ Standard loss functions exist for each type of prediction
 - Regression
 - Classification

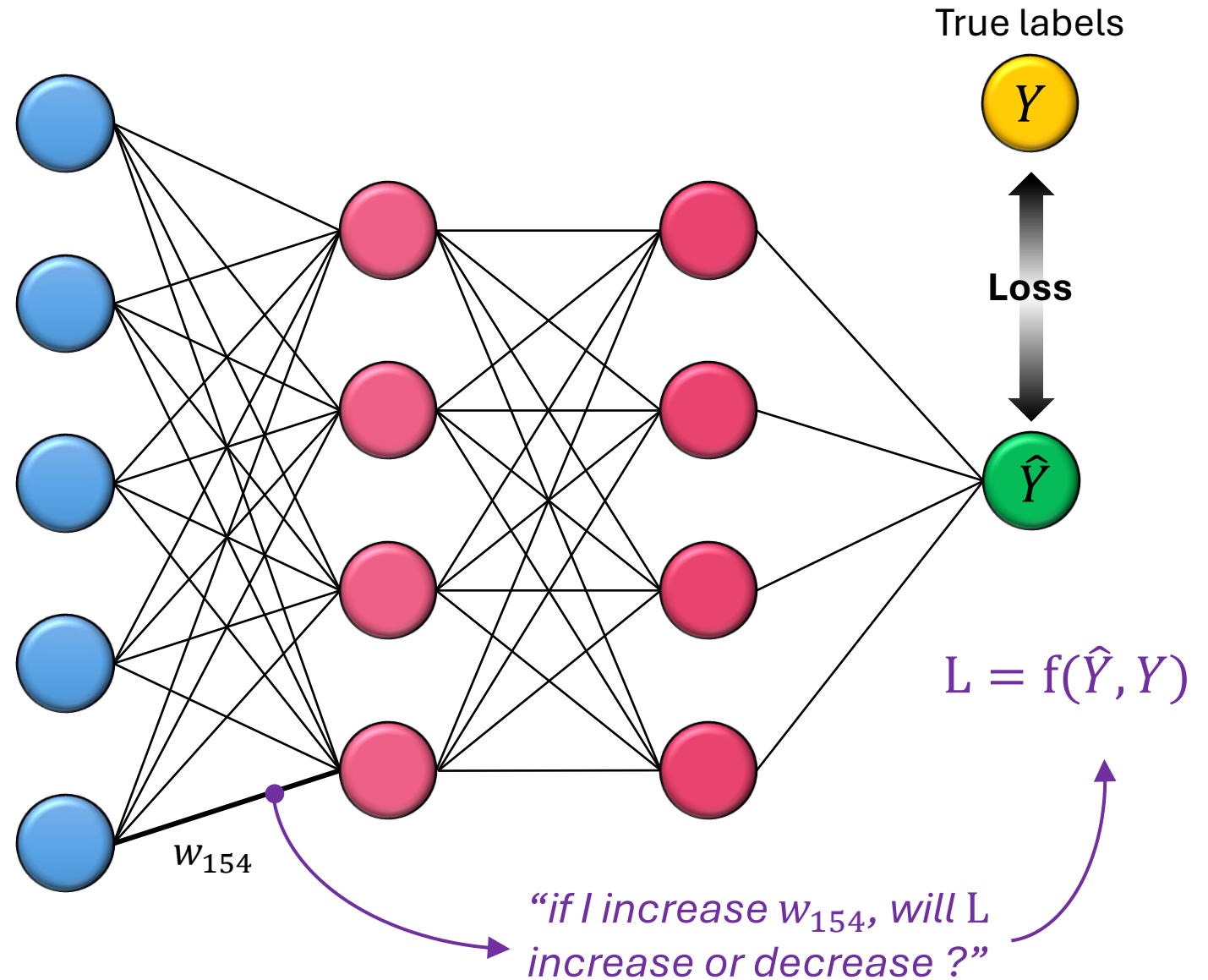


- ❑ The loss function guides learning. Without it, we can't know if the model is improving
- ❑ During training, loss must decrease, otherwise the model is not learning
- ❑ **The goal of learning process is to minimize this loss**



4 Backpropagation

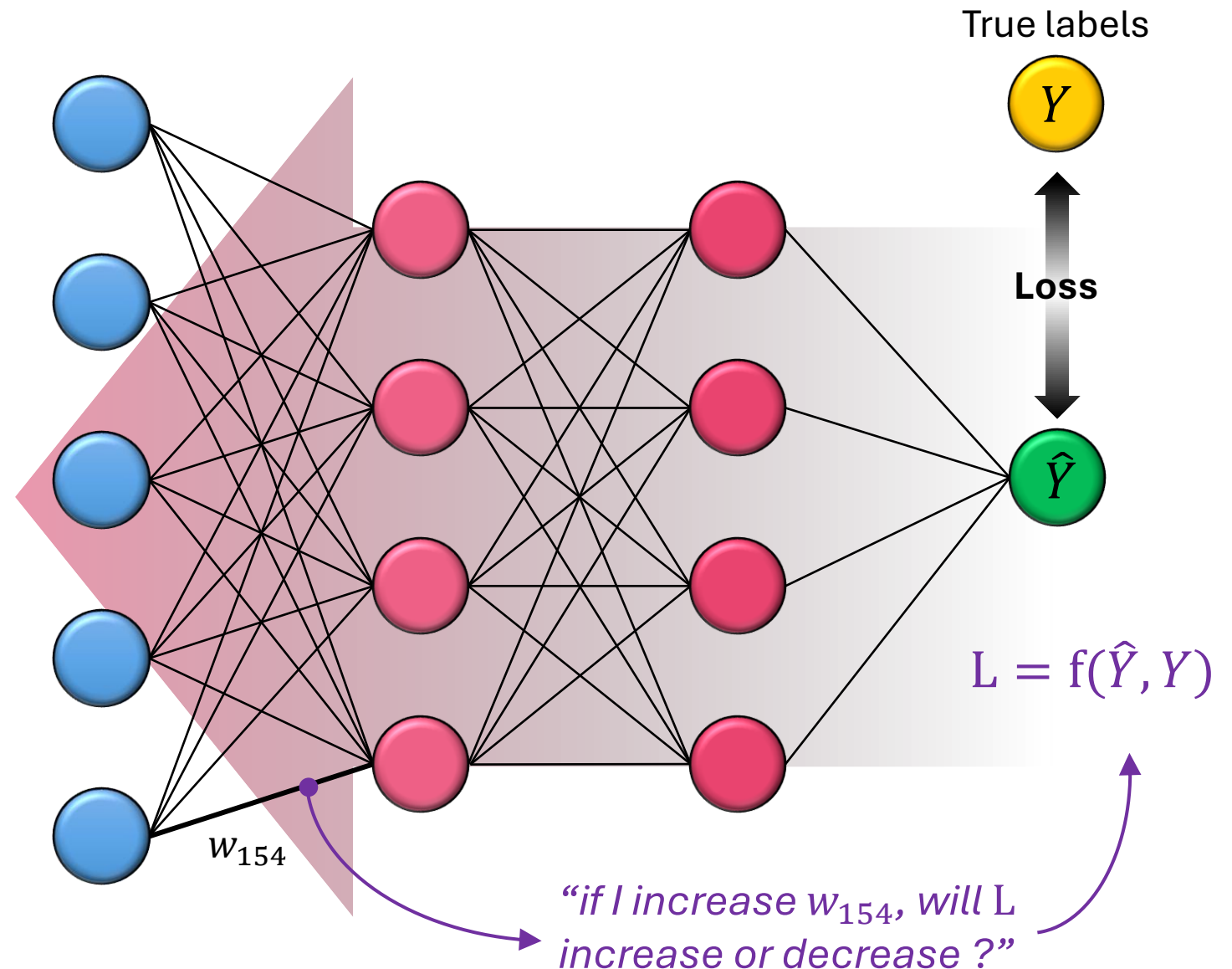
- ❑ To minimize the loss, we need to know how the loss changes with respect to each parameter



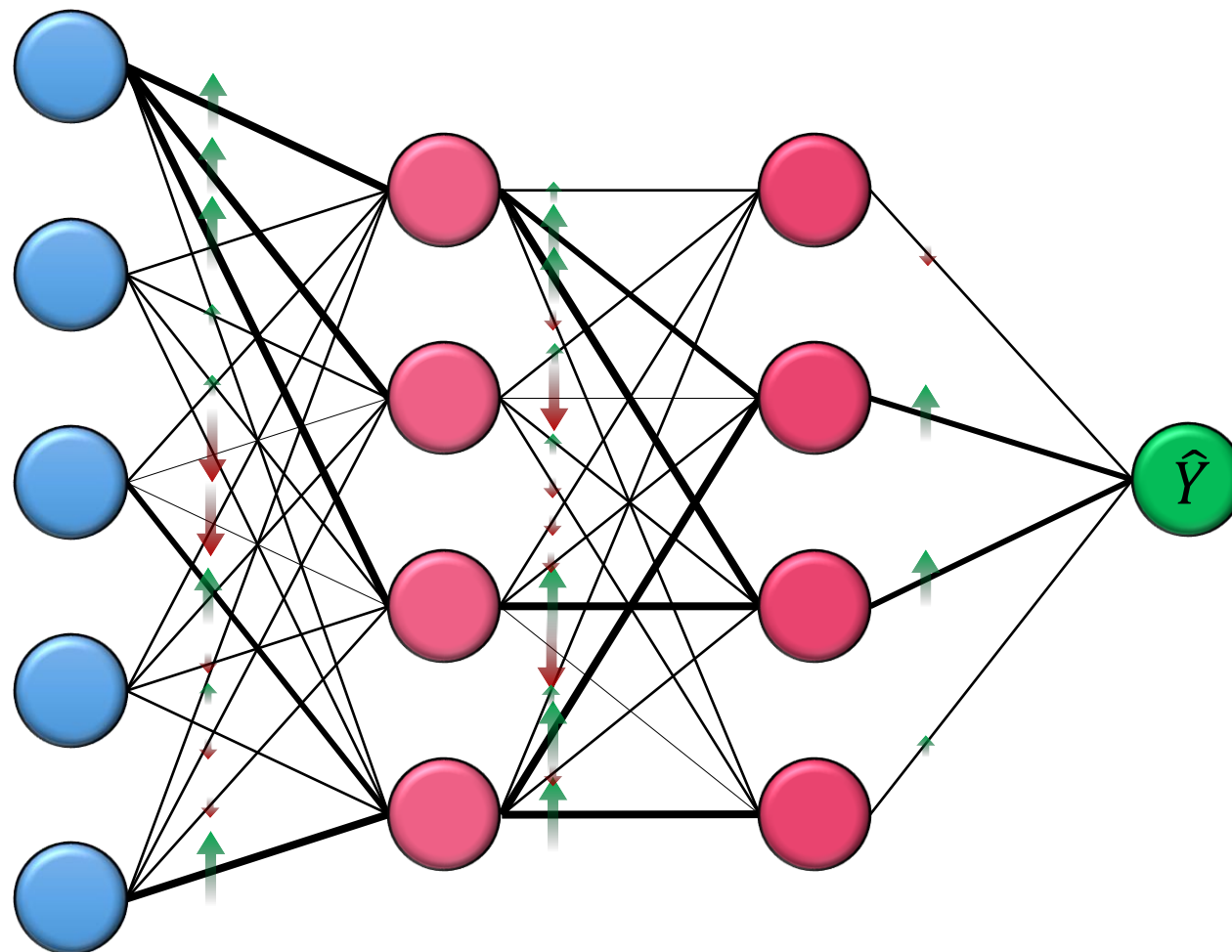
4

Backpropagation

- ❑ To minimize the loss, we need to know how the loss changes with respect to each parameter
- ❑ To do that, **backpropagation** automatically calculates the gradients (partial derivatives) of the loss function with respect to each model parameter, working backward from the output layer
- ❑ These gradients indicate how to adjust weights to reduce the loss

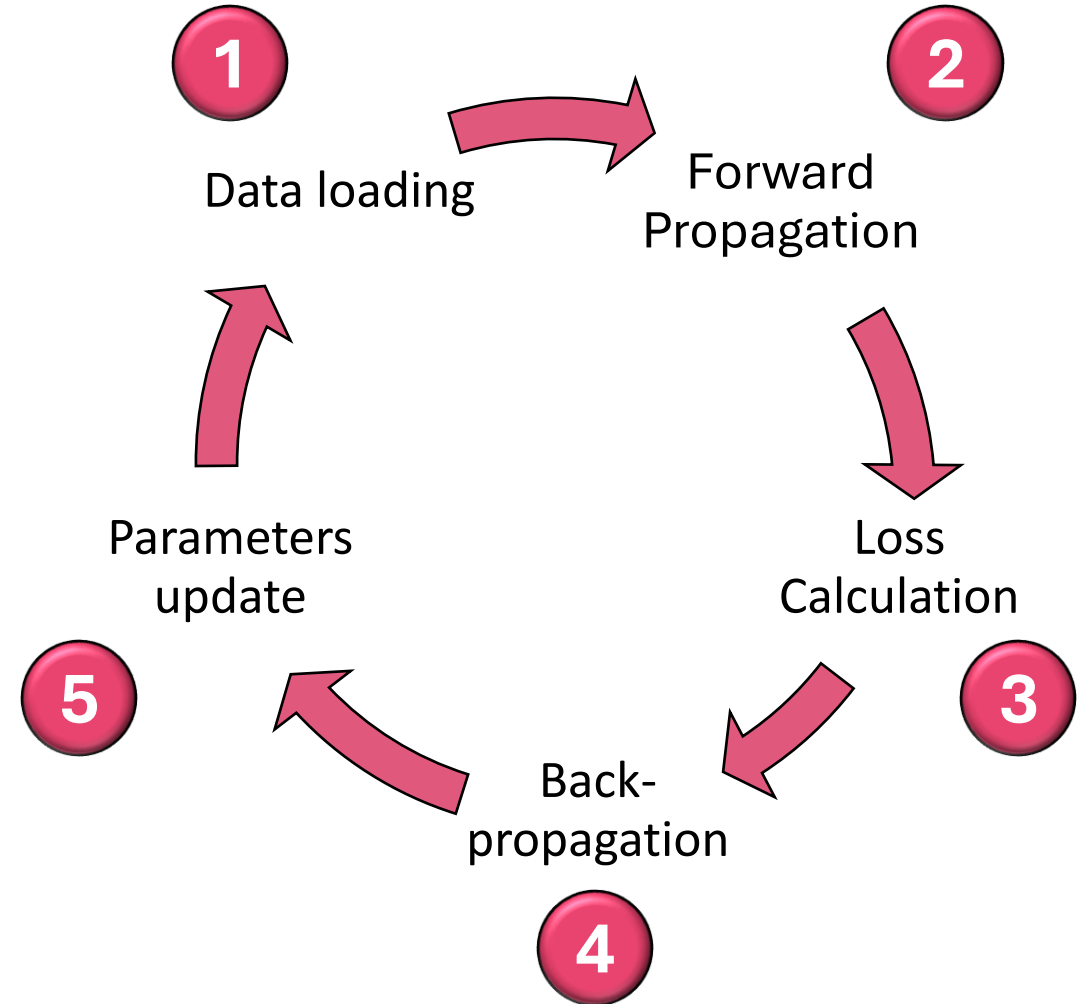


- ❑ Finally, an algorithm called **optimizer** updates each model parameter using the computed gradients, in order to minimize the loss
- ❑ Loss should now decrease
- ❑ Research teams continuously propose new optimizers, using different updating rules, to improve:
 - Their speed
 - Their accuracy



The training cycle

- ❑ Step 1 to 5 are repeated until loss reaches an acceptable value or until the assigned number of iterations is reached
- ❑ When the entire training dataset has been passed once through the full training cycle (forward pass, loss, backprop, and optimization), we call it one **epoch**
- ❑ In practice, models are trained for several epochs, so the network gradually learns from the same data multiple times



Loss Functions

Regression Loss

- ❑ **Mean Squared Error (MSE)** is the standard loss function used to train regression models
 - ❑ MSE measures direct distance between truth value and model's prediction (output)
 - ❑ MSE penalizes large errors a lot
-
- ❑ **Root Mean Squared Error (RMSE)** is not used for training but for human interpretation (evaluation). It keeps the unit of y unchanged

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

N : total number of samples

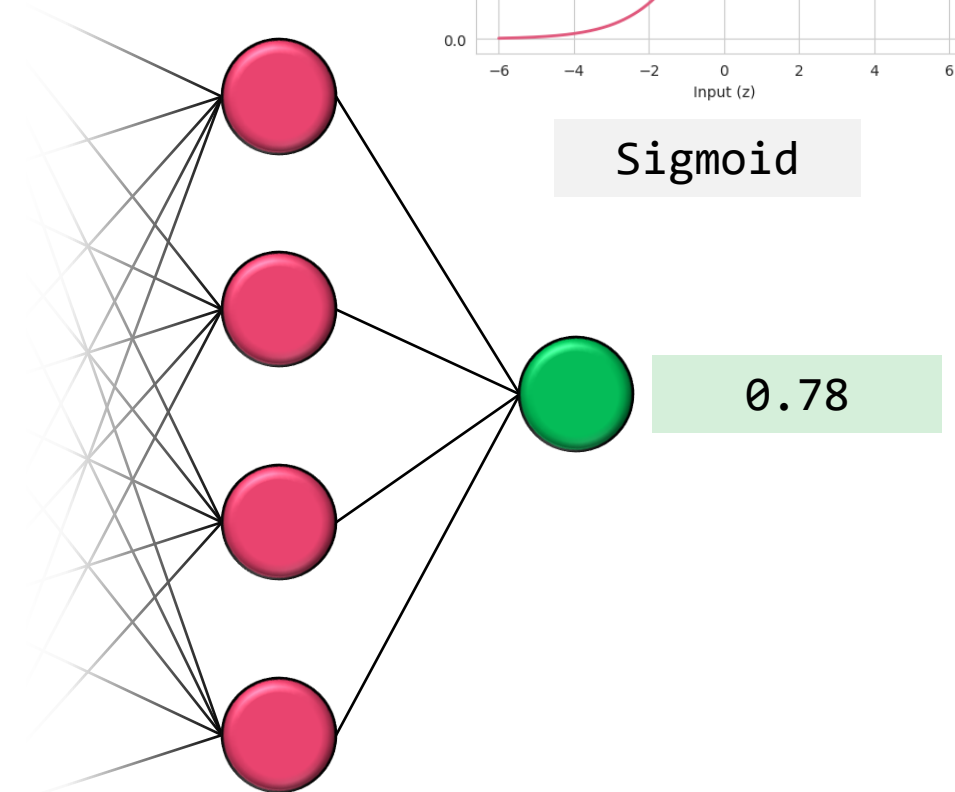
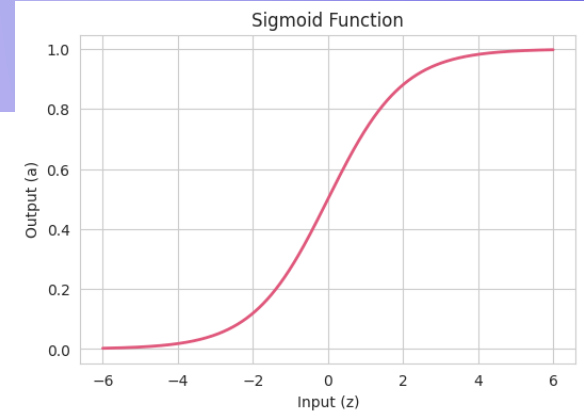
y_i : true value of sample i

$\hat{y}_i$: predicted value of sample i

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Binary Classification Loss

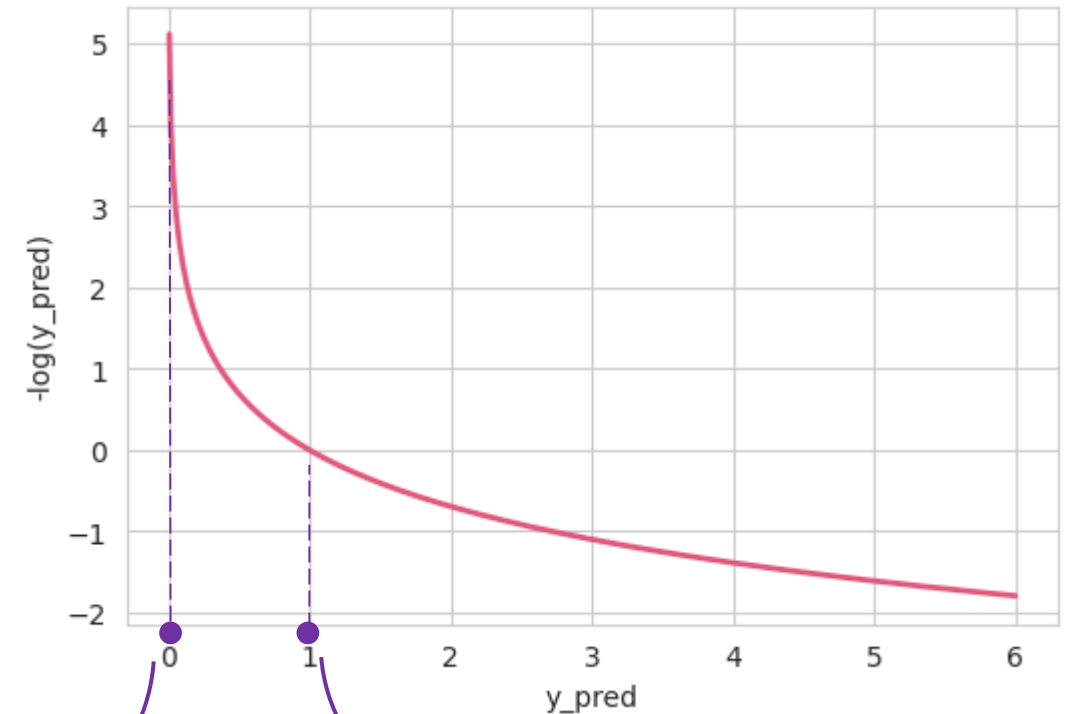
- ❑ Reminder:
 - A Binary Classifier output layer uses Sigmoid activation function
 - Predicted (output) value ($\hat{y}$) is a probability value (between 0 and 1)
- ❑ True labels (y) have only two possible values: 0 or 1
- ❑ Error will be thus computed by comparing a probability to 0 or 1



Binary Classification Loss

- ❑ **Binary Cross Entropy (BCE)** is the loss used for binary classification
- ❑ BCE is also called **Log Loss**
- ❑ BCE uses (negative) Natural Logarithm function to handle both cases:
 - When $y_i = 1$
 - When $y_i = 0$

$$Loss_{y_i=1} = -\log(\hat{y}_i)$$



*If $\hat{y}_i = 0$:
loss is maximal (∞)*

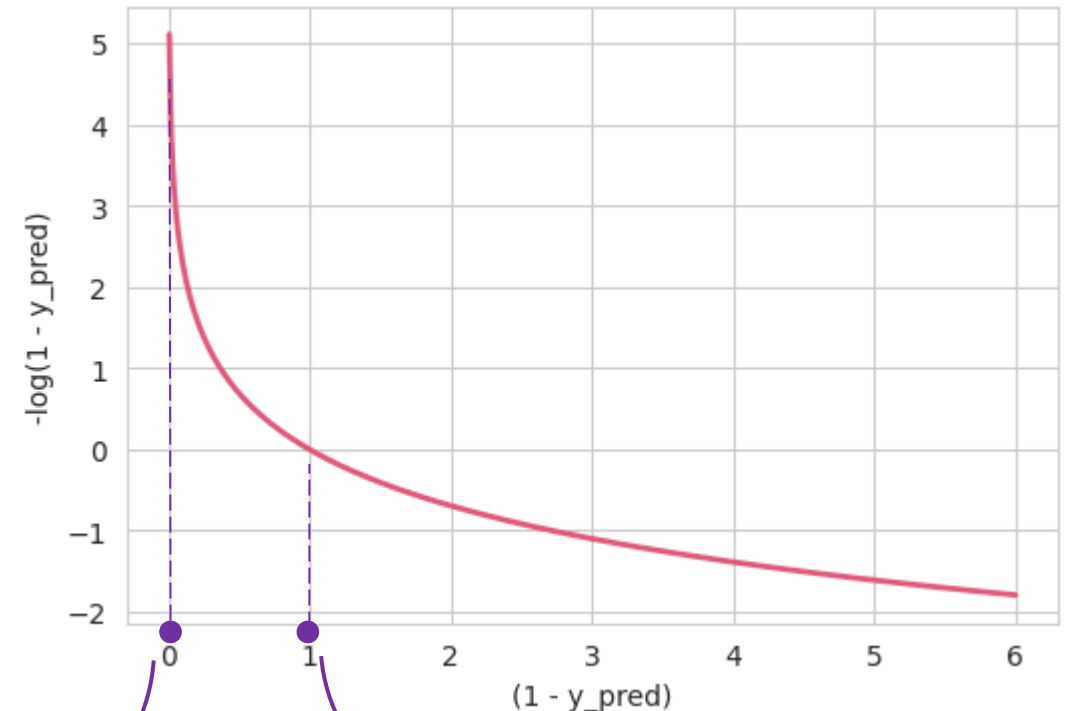
*If $\hat{y}_i = 1$:
loss = 0*

Binary Classification Loss

- ❑ **Binary Cross Entropy (BCE)** is the loss used for binary classification
- ❑ BCE is also called **Log Loss**
- ❑ BCE uses (negative) Natural Logarithm function to handle both cases:
 - When $y_i = 1$
 - When $y_i = 0$



$$Loss_{y_i=0} = -\log(1 - \hat{y}_i)$$



If $\hat{y}_i = 1$:
 $(1 - \hat{y}_i) = 0$,
loss is maximal (∞)

If $\hat{y}_i = 0$:
 $(1 - \hat{y}_i) = 1$,
loss = 0

Binary Classification Loss

- ❑ Binary Cross Entropy for 1 sample i :

$$BCE_i = - \underbrace{(y_i \cdot \log(\hat{y}_i))}_{\substack{\text{part for} \\ y_i = 1}} + \underbrace{(1 - y_i) \cdot \log(1 - \hat{y}_i)}_{\substack{\text{part for} \\ y_i = 0}}$$

y_i : true value of sample i
 $\hat{y}_i$: predicted value of sample i

Binary Classification Loss

- Binary Cross Entropy for 1 sample i :

$$BCE_i = - \underbrace{(y_i \cdot \log(\hat{y}_i))}_{\substack{\text{part for} \\ y_i = 1}} + \underbrace{(1 - y_i) \cdot \log(1 - \hat{y}_i)}_{\substack{\text{part for} \\ y_i = 0}}$$

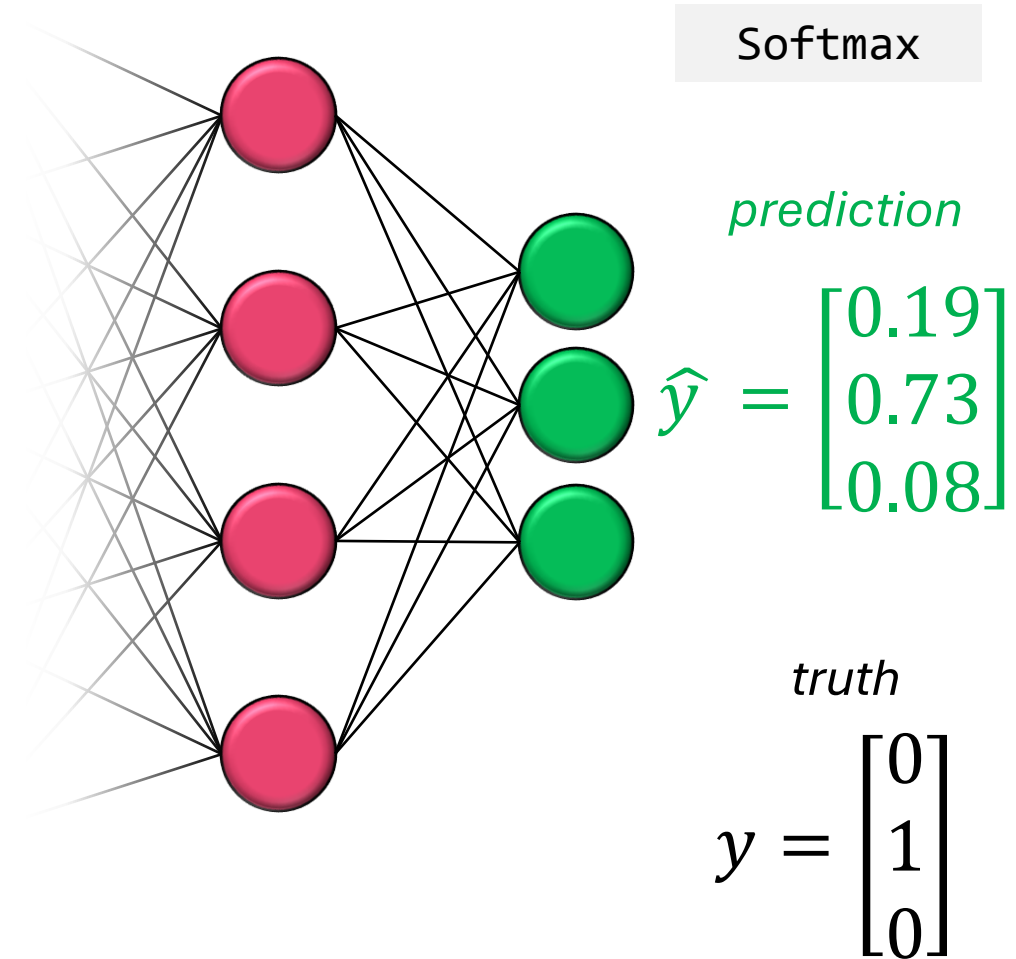
- Binary Cross Entropy for all samples:

$$BCE = -\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

N : total number of samples
 y_i : true value of sample i
 $\hat{y}_i$: predicted value of sample i

Multiclass Classification Loss

- ❑ Reminder:
 - A Multiclass Classifier output layer uses Softmax activation function
 - Predicted (output) value for category j ($\hat{y}_j$) is a probability value (between 0 and 1)
- ❑ True labels for each category (y_j) have only two possible values: 0 or 1. Only one category has a label = 1
- ❑ Error will be thus computed in the same way as for Binary Cross Entropy, but summed for all categories



Multiclass Classification Loss

❑ Categorical Cross Entropy :

$$CCE = \sum_{j=1}^K BCE_j$$

K : total number of categories

N : total number of samples

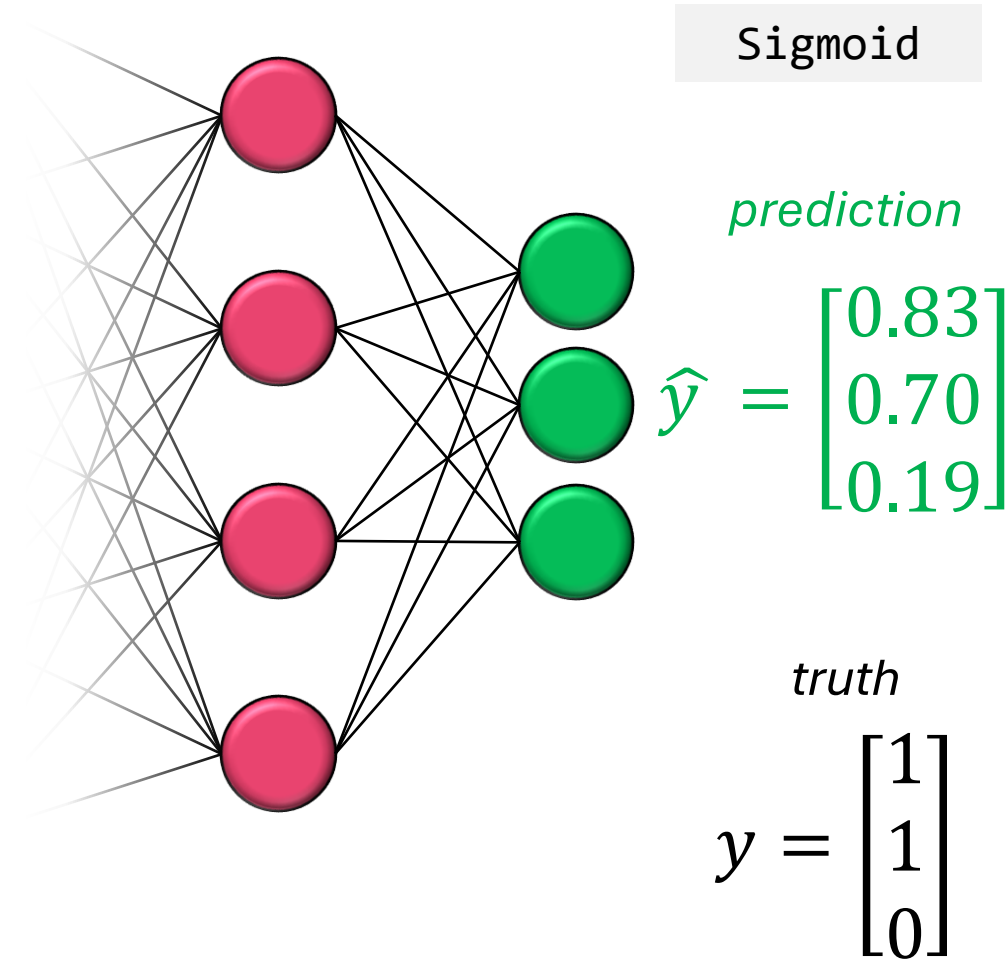
y_i : true value of sample i

ŷ_i : predicted value of sample i

$$CCE = -\frac{1}{N} \sum_{j=1}^K \sum_{i=1}^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

Multilabel Classification Loss

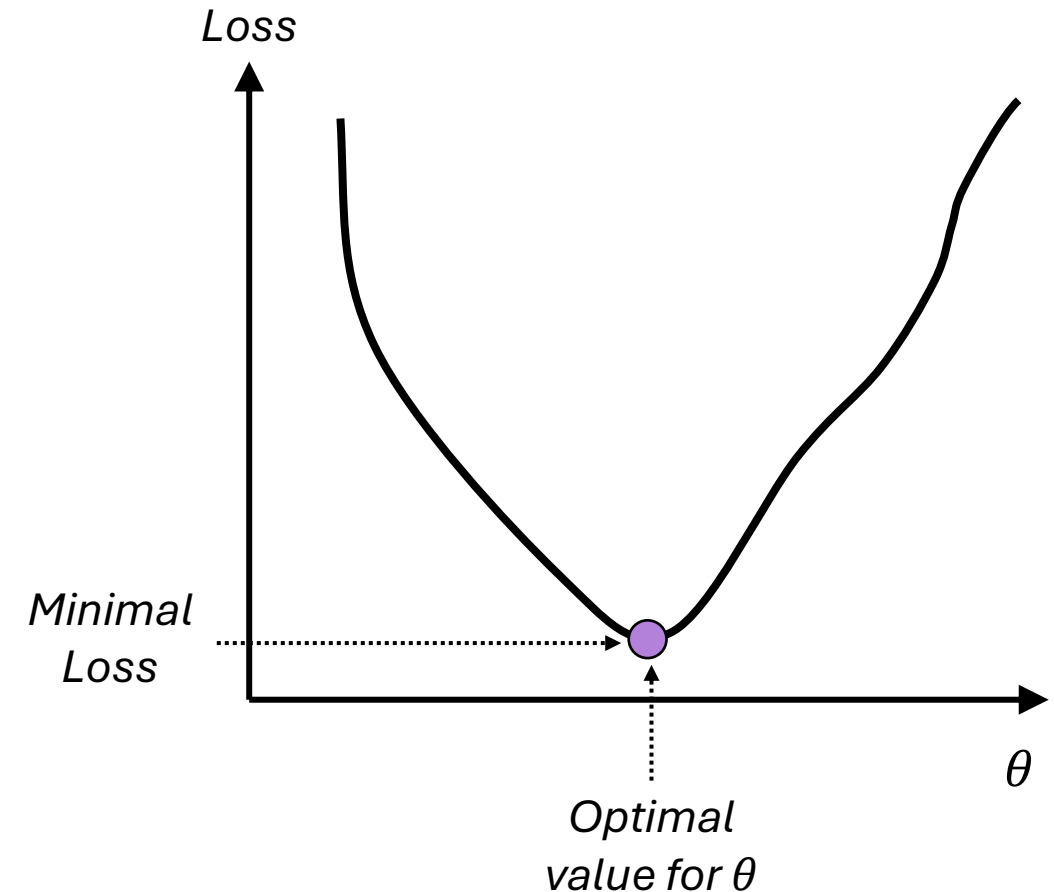
- ❑ Reminder:
 - A Multilabel Classifier output layer uses Sigmoid activation function
 - Predicted (output) value for category j ($\hat{y}_j$) is a probability value (between 0 and 1)
- ❑ True labels for each category (y_j) have only two possible values: 0 or 1. Each of them are independent
- ❑ Error will be thus computed exactly as for Multiclass Classifier, with Categorical Cross Entropy



The Gradient Descent

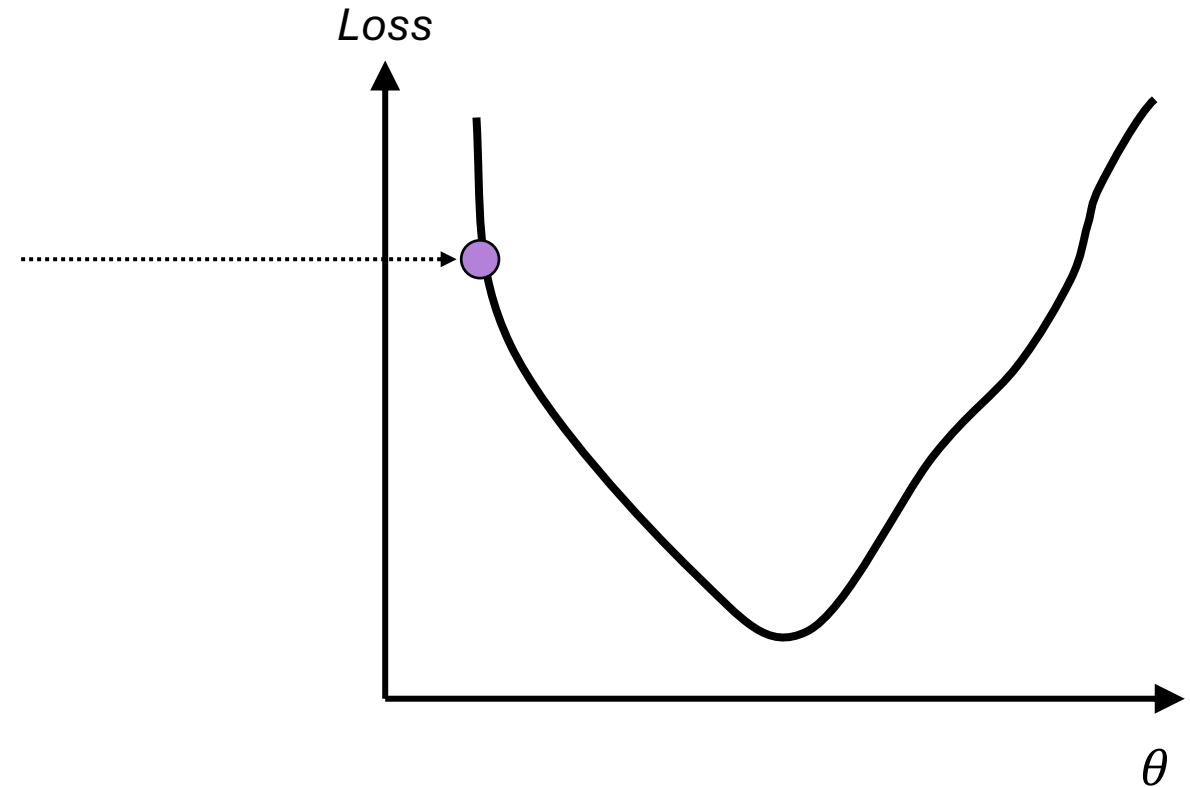
Gradient Descent

- ❑ During training, we want to find the optimal value for each parameter θ (e.g. a weight) so that Loss is minimal
- ❑ For that we could compute Loss for every possible θ value but it is not affordable (too much compute)
- ❑ We rather use **Gradient descent** which is a more efficient method



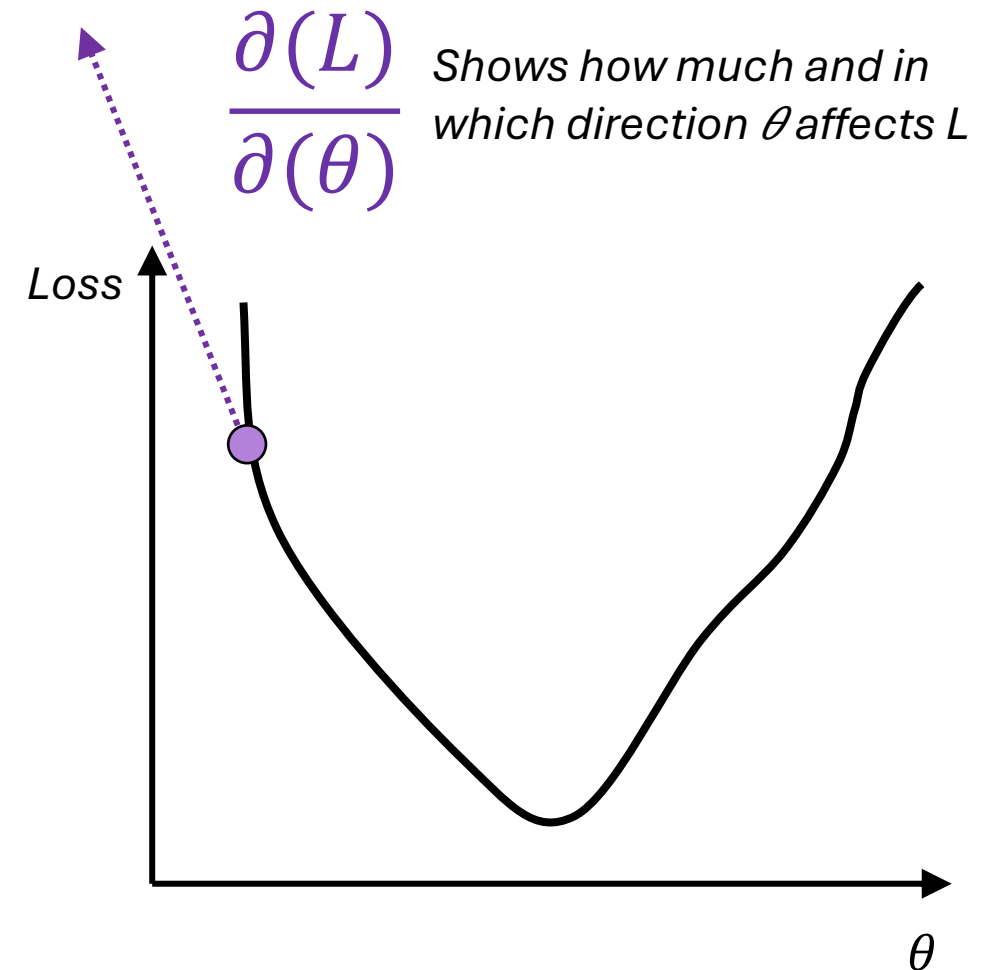
Gradient Descent

- ❑ Gradient Descent is an iterative mathematical method to find the minimum of a (loss) function
- ❑ Gradient Descent works in 3 steps
 - **Step 1: parameter value is taken (randomly initialized at first)**



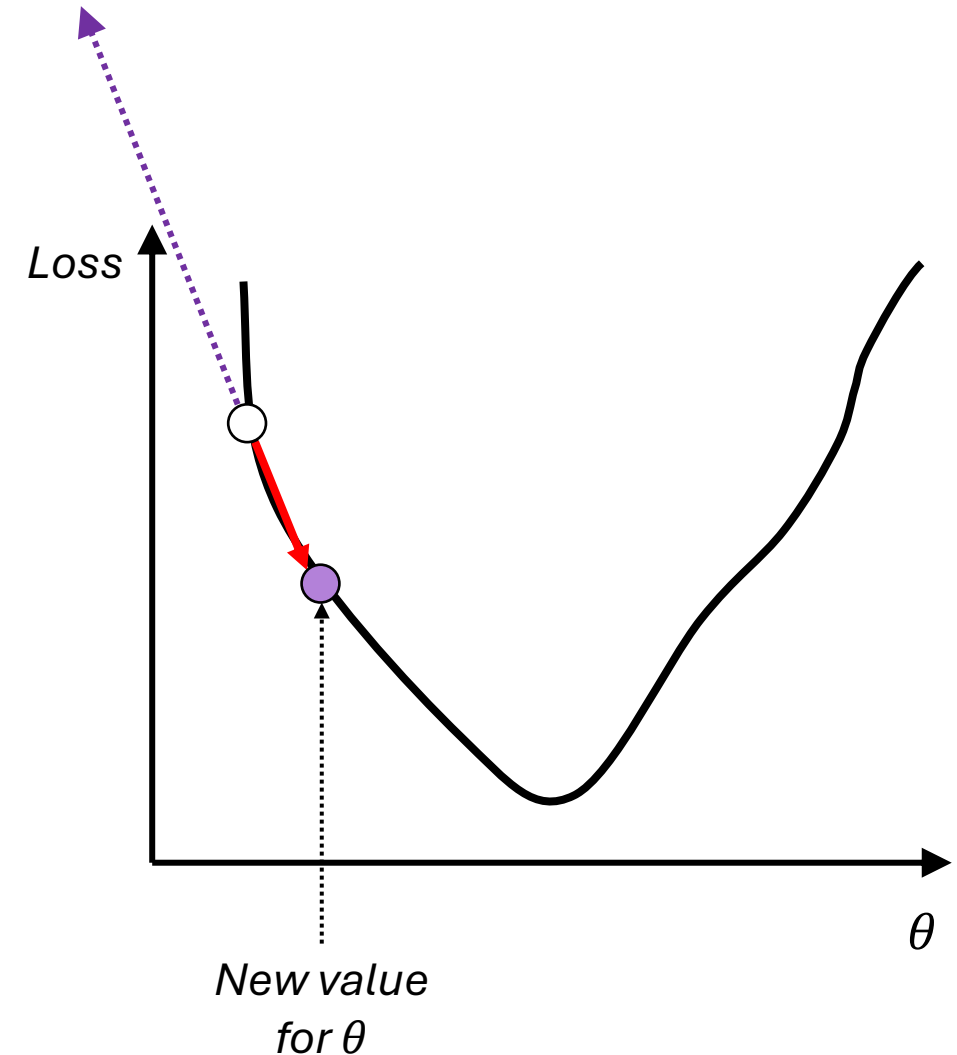
Gradient Descent

- ❑ Gradient Descent is an iterative mathematical method to find the minimum of a (loss) function
- ❑ Gradient Descent works in 3 steps
 - Step 1: parameter value is taken (randomly initialized at first)
 - **Step 2: gradient of the Loss function (partial derivative of Loss w/r to θ) is computed**



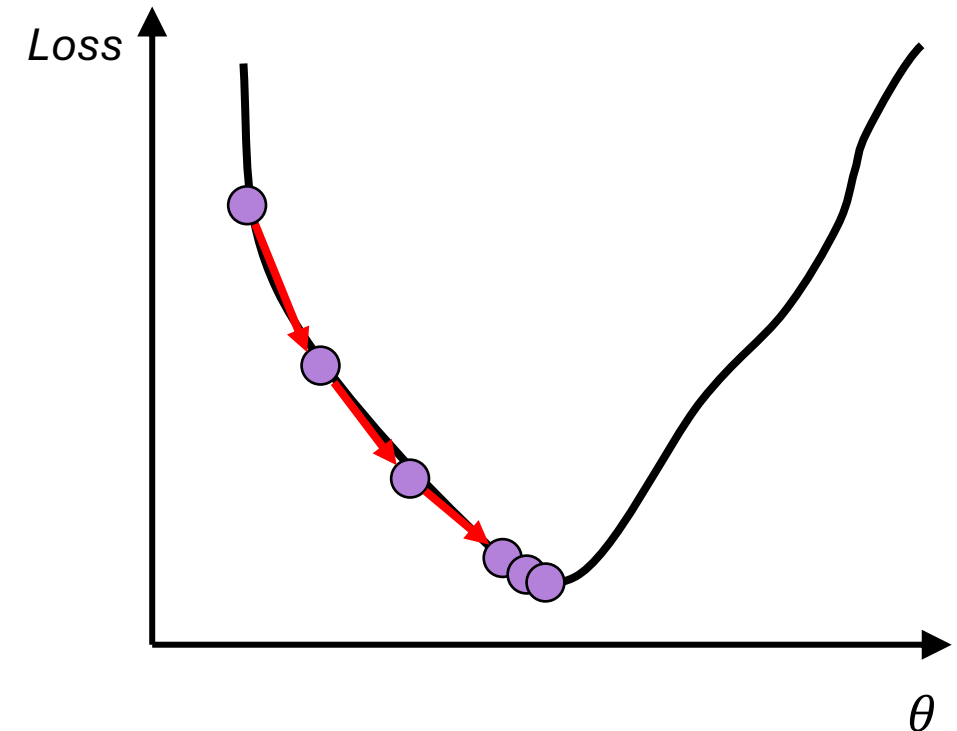
Gradient Descent

- ❑ Gradient Descent is an iterative mathematical method to find the minimum of a (loss) function
- ❑ Gradient Descent works in 3 steps
 - Step 1: parameter value is taken (randomly initialized at first)
 - Step 2: gradient of the Loss function (partial derivative of Loss w/r to θ) is computed
 - **Step 3: Update θ by taking a step in the opposite direction of the gradient**



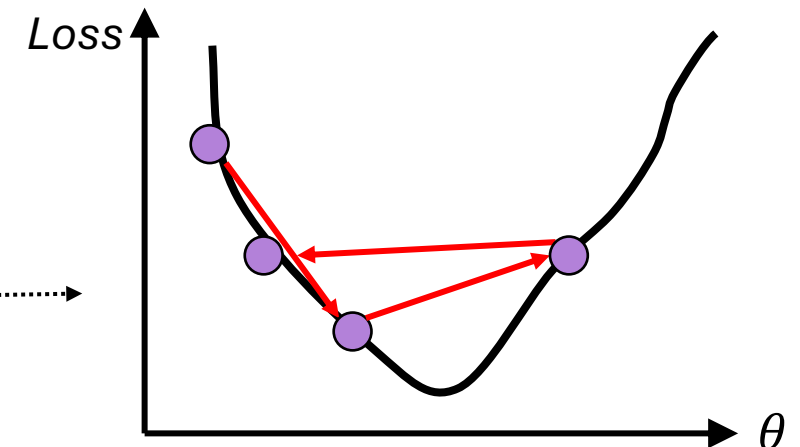
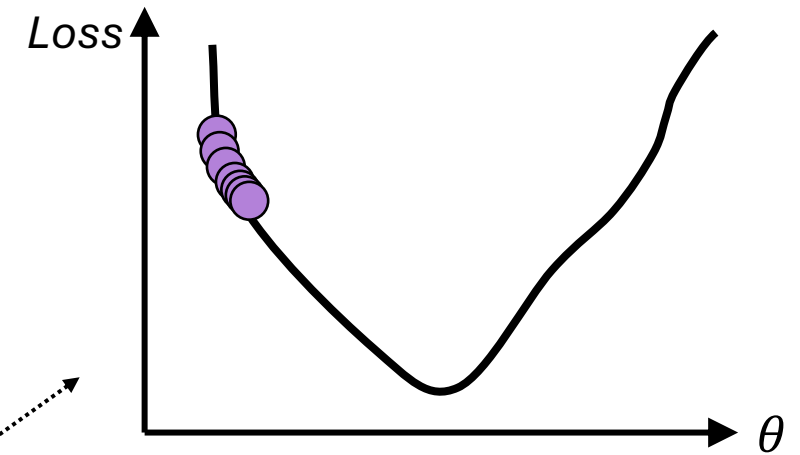
Gradient Descent

- ❑ Steps are large at the beginning, and get smaller as θ gets closer to the optimal value
- ❑ Gradient Descent iterates over and over, until
 - A preset minimum step size is reached (e.g. 0,001)
 - A preset maximum number of iterations if reached (e.g. 500)



Gradient Descent

- ❑ **Learning Rate** (η) is a scalar value that determines the size of the step during Gradient Descent
- ❑ Learning Rate is an **hyperparameter**, a parameter set before training, not learned
- ❑ Learning Rate value is very impactful on the learning process. It has to be chosen carefully
 - If η is too low: learning is too long, and can stop improving
 - If η is too high: θ will overshoot the minimum and never reach it



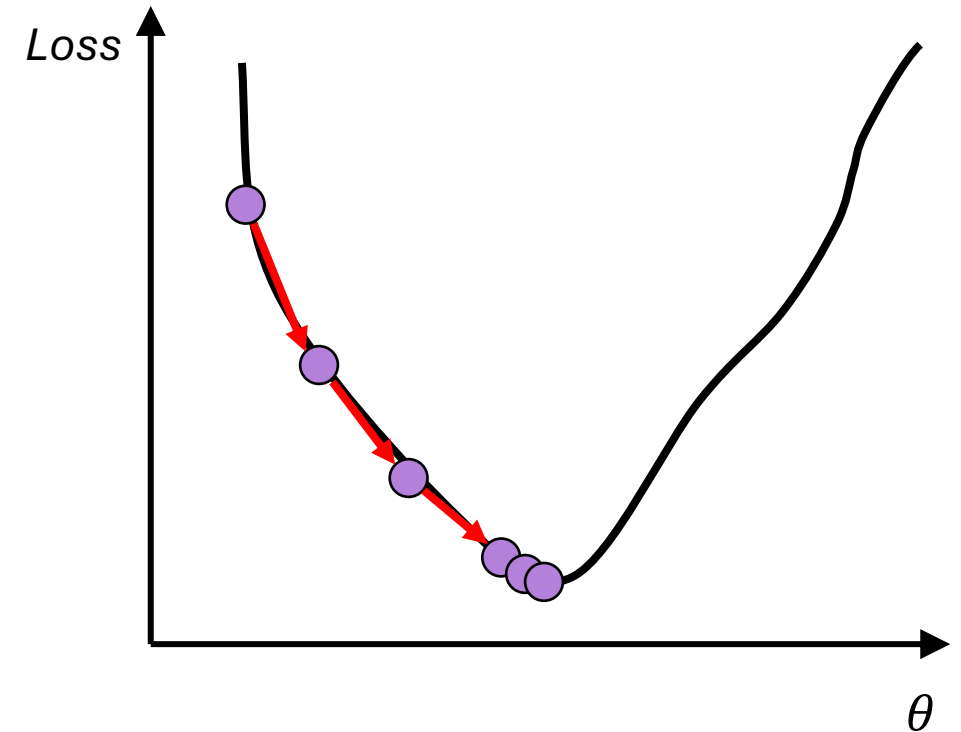
Gradient Descent

- ❑ Vanilla Gradient Descent formula for **any** parameter of the model:

$$\theta_{t+1} = \theta_t - \underbrace{\eta \cdot \frac{\partial(L)}{\partial(\theta_t)}}_{\text{Step size}}$$

New value Old value Step size

θ_t : 1 parameter at time t
 L : loss function
 η : learning rate



Gradient Descent

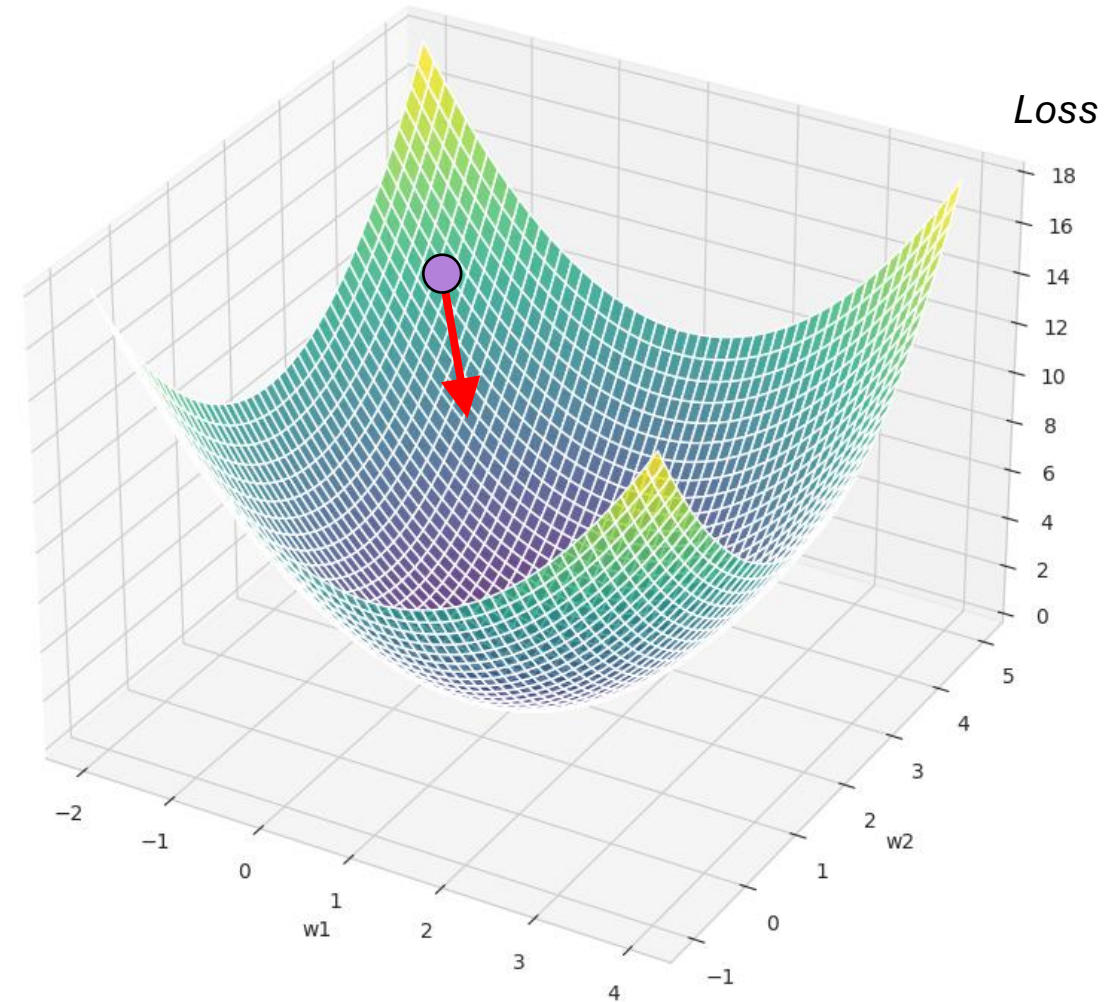
- ❑ Vanilla Gradient Descent formula for **all** parameters of the model:

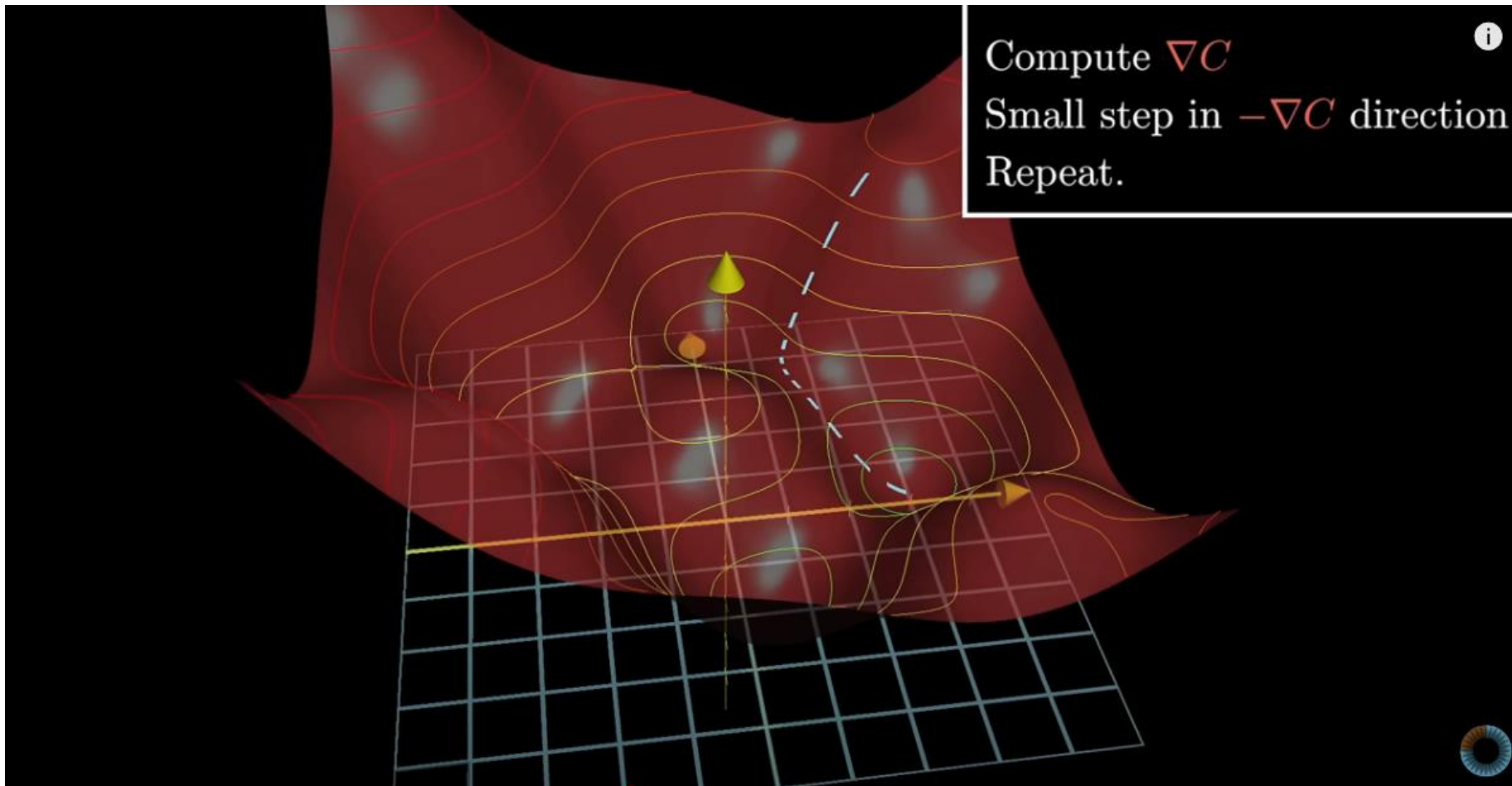
$$\theta_{t+1} = \theta_t - \underbrace{\eta \cdot \nabla L(\theta_t)}_{\text{Step size}}$$

New value θ_{t+1} , Old value θ_t , Gradient (vector) $\nabla L(\theta_t)$, Step size $\eta \cdot \nabla L(\theta_t)$

θ_t : vector of all parameters at time t
 L : loss function
 η : learning rate

Loss surface for a model with 2 parameters





For a more visual explanation of gradient descent, check this [3 Bue 1 Brown video](#)

The Backpropagation

The Backpropagation

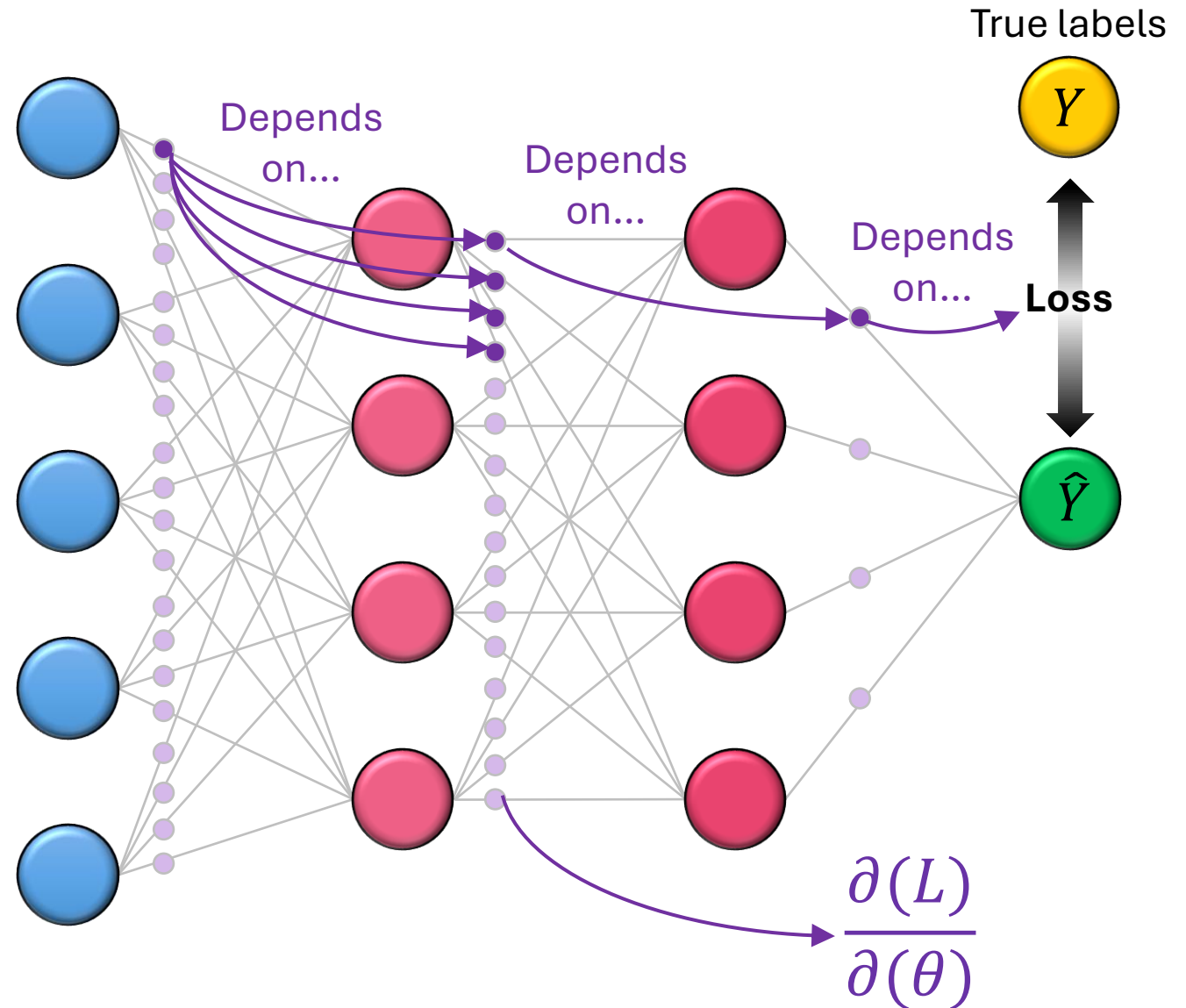
- ❑ Gradient vector ∇L generalizes the idea of a derivative to functions with multiple parameters
- ❑ In machine learning, the gradient contains all the **partial derivatives** of the loss function with respect to each parameter (weight, bias, etc.)
- ❑ The **direction** of the gradient shows how to increase the loss most rapidly, and its **magnitude** tells how steep that increase is
- ❑ During optimization, we move in the opposite direction of the gradient to reduce the loss

$$\nabla L = \begin{bmatrix} \frac{\partial(L)}{\partial(\theta_1)} \\ \frac{\partial(L)}{\partial(\theta_2)} \\ \dots \\ \frac{\partial(L)}{\partial(\theta_n)} \end{bmatrix}$$

L : loss function
 θ_i : parameter i

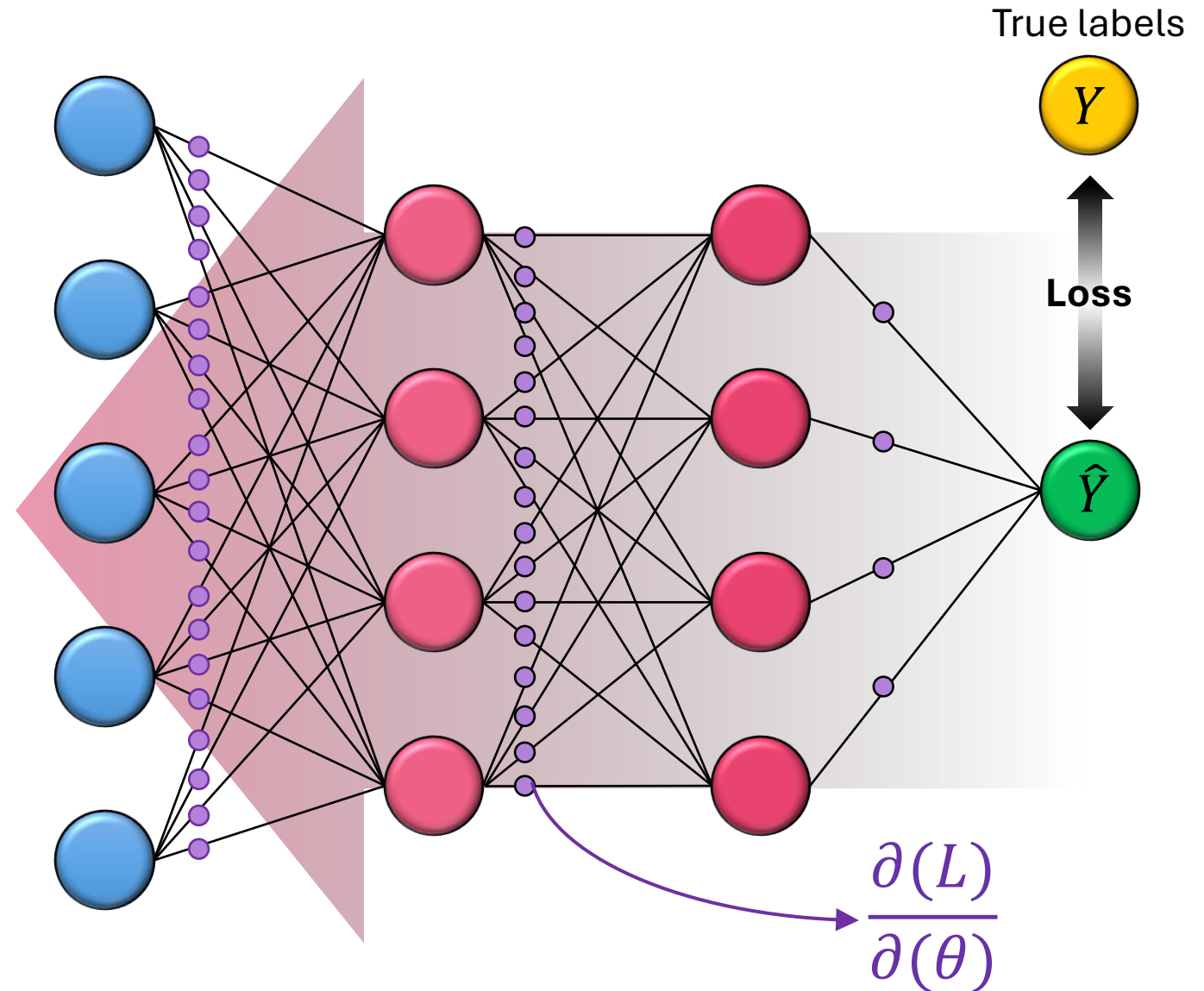
The Backpropagation

- ❑ Problem: *Loss* depends on many “nested” operations (layers, activations, weights...)
- ❑ Consequently, the partial derivative of each parameter depends on the derivatives of the parameters that come after it in the network



The Backpropagation

- ❑ Problem: *Loss* depends on many “nested” operations (layers, activations, weights...)
- ❑ Consequently, the partial derivative of each parameter depends on the derivatives of the parameters that come after it in the network
- ❑ To compute how much each parameter contributes to the loss, we must propagate these derivatives backward through the network
- ❑ This process is called **backpropagation**



How to compute gradients?

- ❑ Solution: to calculate the gradient with respect to each of the parameters, we use the **Chain Rule**

- ❑ If: $L = f(g(x))$

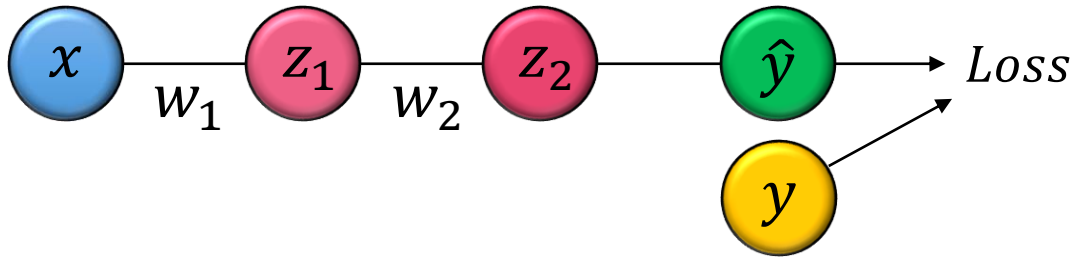
- ❑ Then: $\frac{d(f)}{d(x)} = \frac{d(f)}{d(g)} \cdot \frac{d(g)}{d(x)}$



“Derivative of the outside times derivative of the inside.”

How to compute gradients?

- Example of chain rule application with a tiny network (no bias, no activation, loss = MSE):



$$z_1 = w_1 \cdot x$$

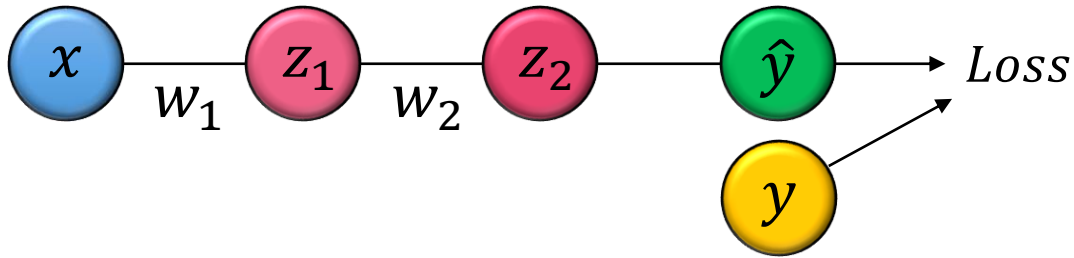
$$z_2 = w_2 \cdot z_1$$

$$\hat{y} = z_2$$

$$L = (\hat{y} - y)^2$$

How to compute gradients?

- Example of chain rule application with a tiny network (no bias, no activation, loss = MSE):



$$z_1 = w_1 \cdot x$$

$$z_2 = w_2 \cdot z_1$$

$$\hat{y} = z_2$$

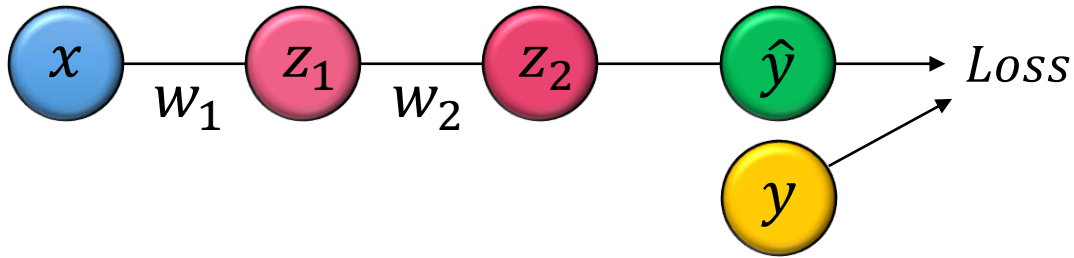
$$L = (\hat{y} - y)^2$$

Chain rule to compute partial derivative of L with respect to w_1 :

$$\frac{\partial(L)}{\partial(w_1)} = \frac{\partial(L)}{\partial(\hat{y})} \cdot \frac{\partial(\hat{y})}{\partial(z_2)} \cdot \frac{\partial(z_2)}{\partial(z_1)} \cdot \frac{\partial(z_1)}{\partial(w_1)}$$

How to compute gradients?

- Example of chain rule application with a tiny network (no bias, no activation, loss = MSE):



$$z_1 = w_1 \cdot x$$

$$z_2 = w_2 \cdot z_1$$

$$\hat{y} = z_2$$

$$L = (\hat{y} - y)^2$$

$$\frac{\partial(L)}{\partial(w_1)} = \frac{\partial(L)}{\partial(\hat{y})} \cdot \frac{\partial(\hat{y})}{\partial(z_2)} \cdot \frac{\partial(z_2)}{\partial(z_1)} \cdot \frac{\partial(z_1)}{\partial(w_1)}$$

$$\frac{\partial(L)}{\partial(\hat{y})} = 2(\hat{y} - y)$$

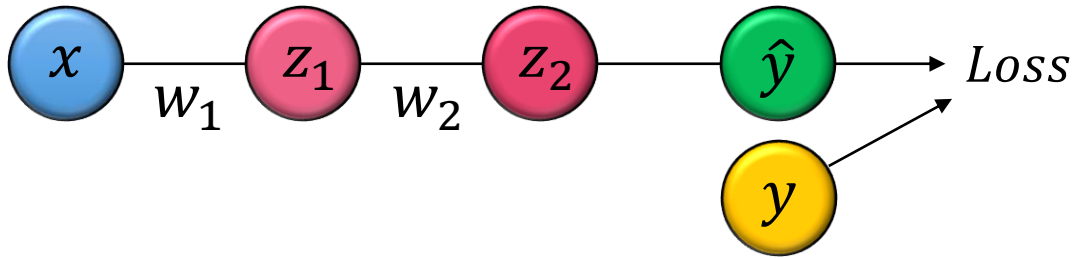
$$\frac{\partial(\hat{y})}{\partial(z_2)} = 1$$

$$\frac{\partial(z_2)}{\partial(z_1)} = w_2$$

$$\frac{\partial(z_1)}{\partial(w_1)} = x$$

How to compute gradients?

- Example of chain rule application with a tiny network (no bias, no activation, loss = MSE):



$$z_1 = w_1 \cdot x$$

$$z_2 = w_2 \cdot z_1$$

$$\hat{y} = z_2$$

$$L = (\hat{y} - y)^2$$

$$\frac{\partial(L)}{\partial(w_1)} = \frac{\partial(L)}{\partial(\hat{y})} \cdot \frac{\partial(\hat{y})}{\partial(z_2)} \cdot \frac{\partial(z_2)}{\partial(z_1)} \cdot \frac{\partial(z_1)}{\partial(w_1)}$$

$$\frac{\partial(L)}{\partial(\hat{y})} = 2(\hat{y} - y)$$

$$\frac{\partial(\hat{y})}{\partial(z_2)} = 1$$

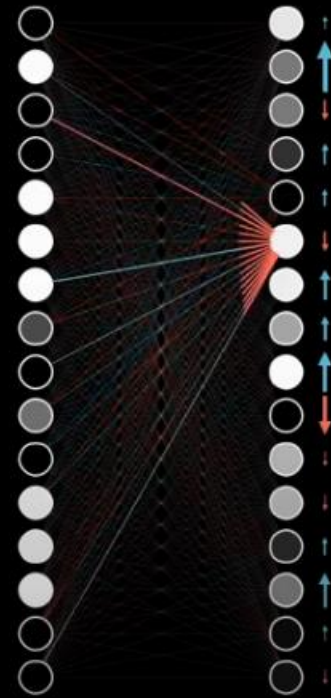
$$\frac{\partial(z_2)}{\partial(z_1)} = w_2$$

$$\frac{\partial(z_1)}{\partial(w_1)} = x$$

$$\frac{\partial(L)}{\partial(w_1)} = 2(\hat{y} - y) \cdot w_2 \cdot x$$



Propagate backwards

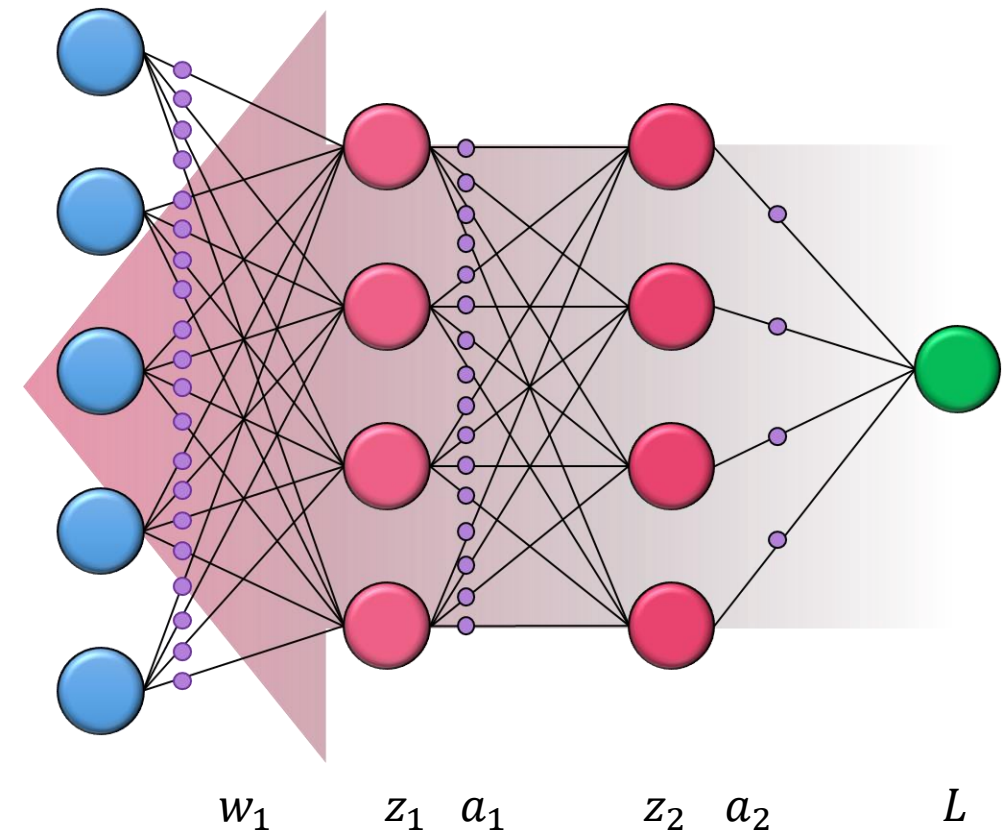


For a more visual explanation of backpropagation, check this [3 Bue 1 Brown video](#)

The Vanishing Gradient Problem

What is Vanishing Gradient Problem?

- ❑ During backpropagation, gradients are propagated backward through layers
- ❑ Each time gradients become small, they vanish as they move toward earlier layers
- ❑ As a result, early layers barely update and model fails to learn deep representations, as if deep networks forget how to train their first layers
- ❑ Before 2011, this has been a major drawback for using NN with many layers



$$\frac{\partial(L)}{\partial(w_1)} = \frac{\partial(L)}{\partial(a_2)} \cdot \frac{\partial(a_2)}{\partial(z_2)} \cdot \frac{\partial(z_2)}{\partial(a_1)} \cdot \frac{\partial(a_1)}{\partial(z_1)} \cdot \frac{\partial(z_1)}{\partial(w_1)}$$

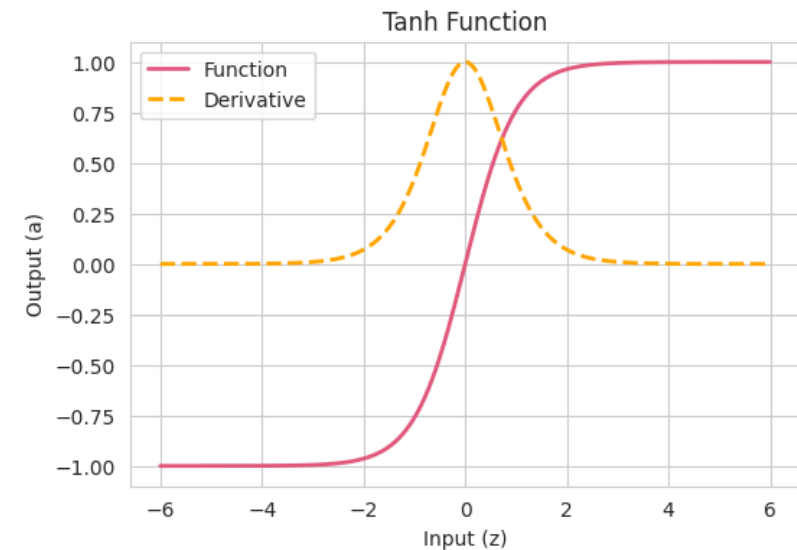
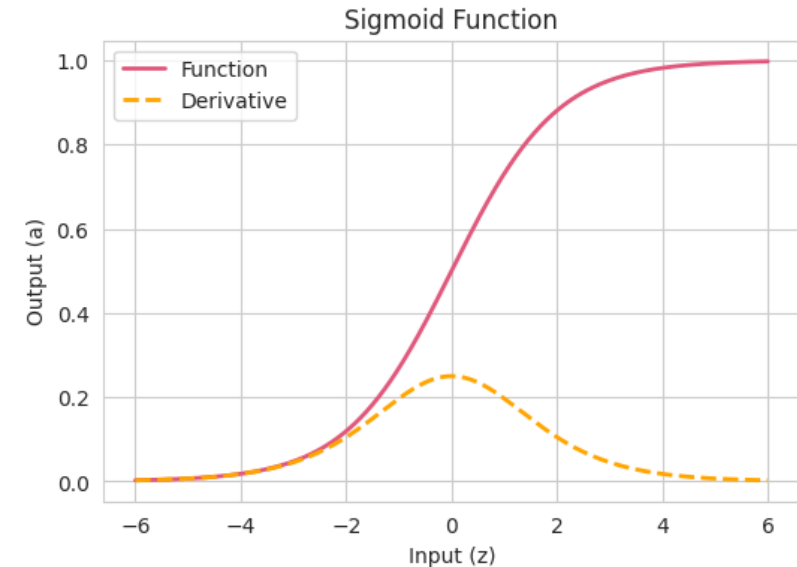
Causes of Vanishing gradients

❑ Activation functions

- Historically, Sigmoid activation was mostly used. But derivatives are very small, even at origin
- Tanh was then preferred, but it also has derivatives < 1 in most regions

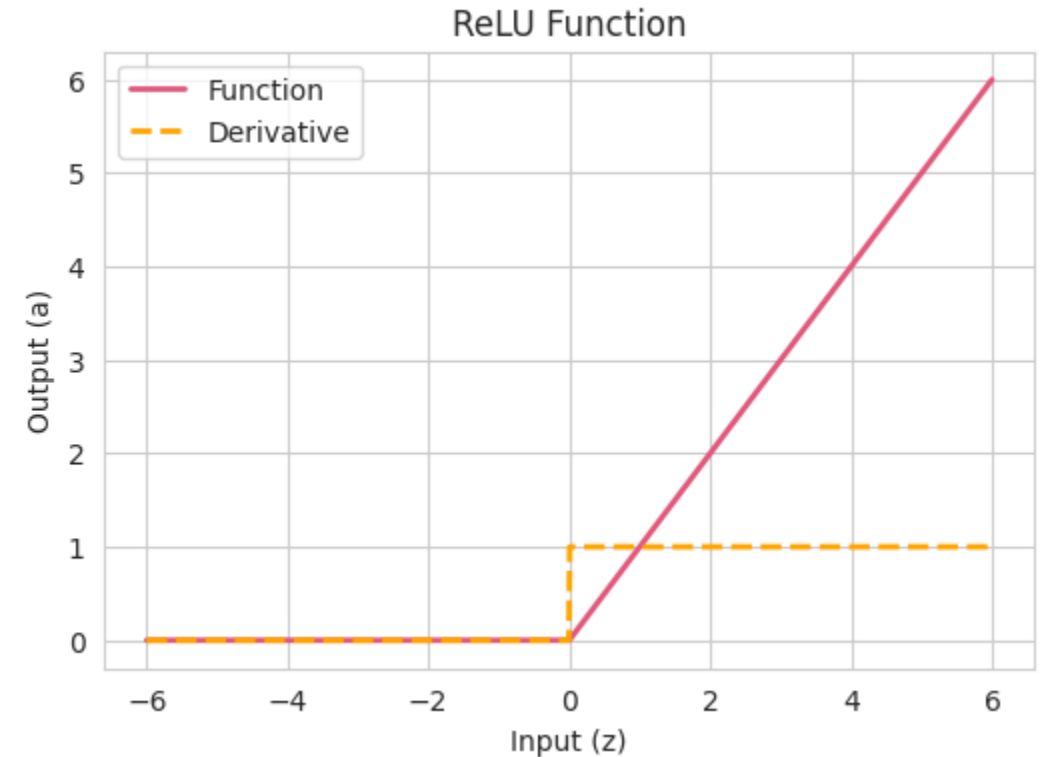
❑ Deepness

- Each layer multiplies gradients by local derivatives (chain rule), and multiplying many small numbers generates exponentially smaller gradients
- ❑ This is even worse if **weights are initialized** with extreme values
- ❑ As a result:
- Early layers learn extremely slowly
 - The network may converge to a poor local minimum or not converge at all



The solution

- ❑ Using non-saturating activation functions, like ReLU, Leaky ReLU, avoids this problem (derivative = 1 if z positive)
- ❑ However, The ReLU unit dies when its input is negative, which might block gradient descent (“Dying ReLU problem”)
- ❑ Normalizing weight initialization also helps to distribute their values around 0, mitigating vanishing gradient



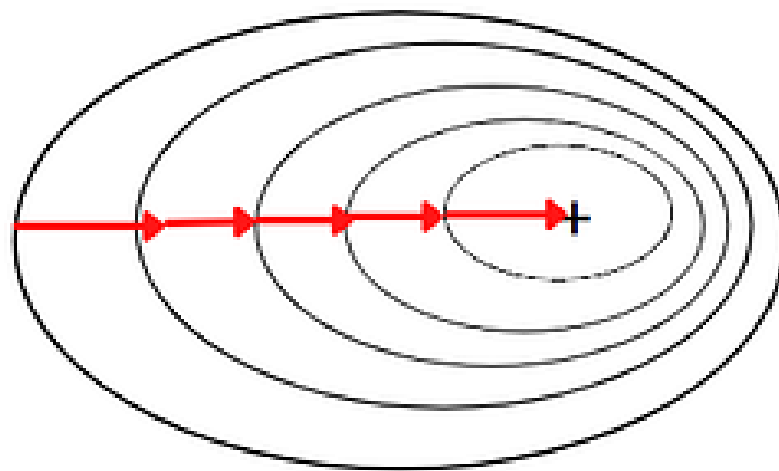
Gradient Descent: Feeding strategies

1

Batch Gradient Descent

- ❑ By default, GD loads the entire training dataset into memory to compute the gradient at each iteration
- ❑ The weight update happens once per epoch

Batch Gradient Descent



1

Batch Gradient Descent

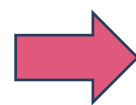
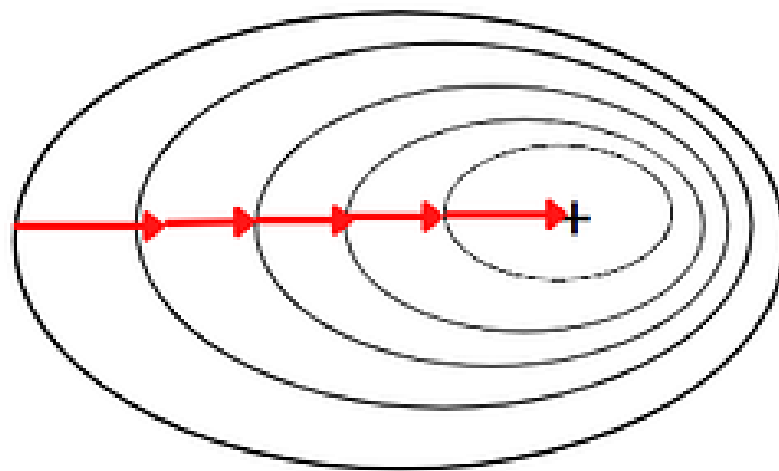
- ❑ By default, GD loads the entire training dataset into memory to compute the gradient at each iteration
- ❑ The weight update happens once per epoch

👍 Very stable convergence (less noisy)

👍 Accurate estimate of the true gradient

👎 Slow and memory-heavy for large datasets

Batch Gradient Descent



Not practical for deep learning on big data
So, other techniques have emerged

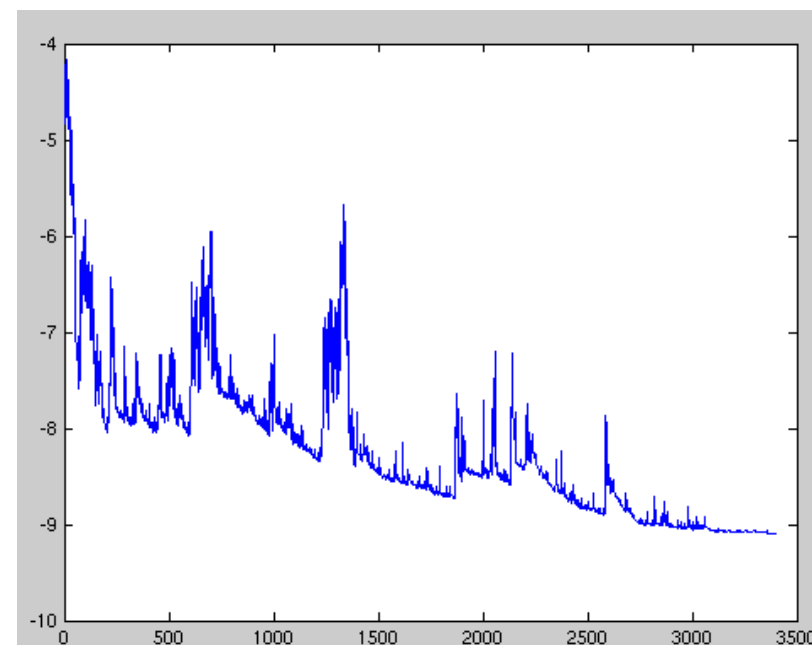
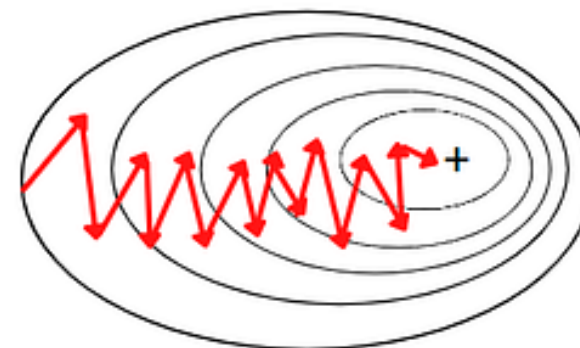
2

Stochastic Gradient Descent



- GD updates parameters using one sample at a time (batch size = 1)

Stochastic Gradient Descent



Convergence plot of stochastic gradient descent — Joe Pharos (2006), CC BY-SA 3.0.



- ❑ GD updates parameters using one sample at a time (batch size = 1)



Very fast updates



Low memory cost (frees memory after each sample)

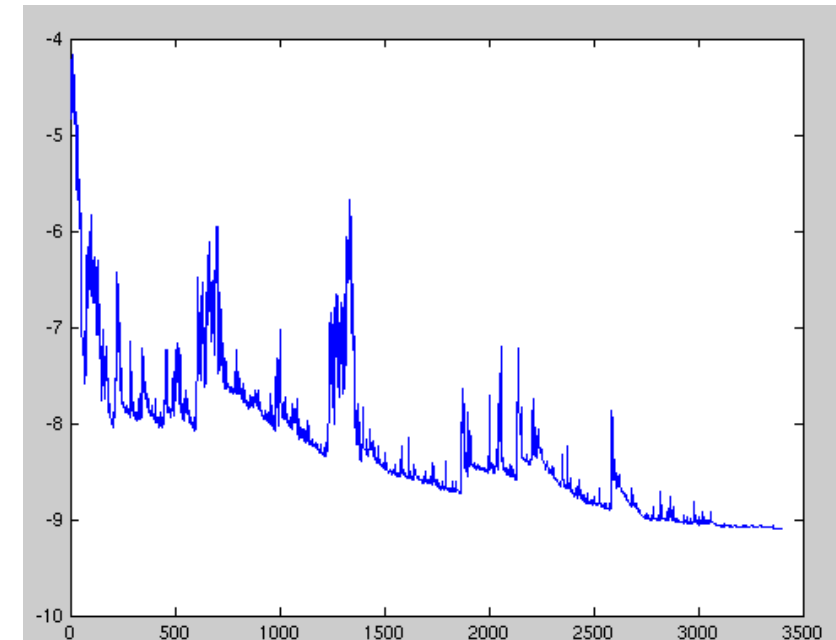
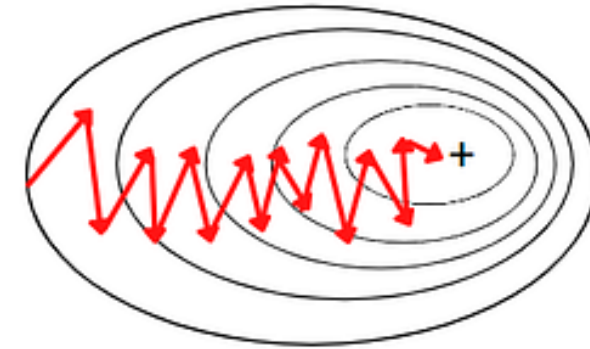


Can generalize better by escaping local minima due to randomness



High variance in updates makes noisy convergence

Stochastic Gradient Descent

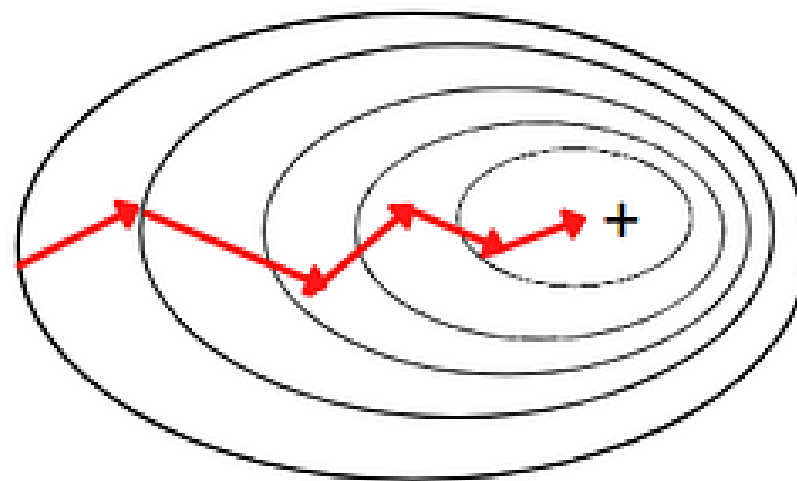


Convergence plot of stochastic gradient descent — Joe Pharos (2006), CC BY-SA 3.0.



- ❑ Compromise between Batch and SGD
- ❑ GD computes gradient on a small batch of samples (e.g. 8, 16, 32, 64...)
- ❑ Most used technique in Deep Learning

Mini-Batch Gradient Descent

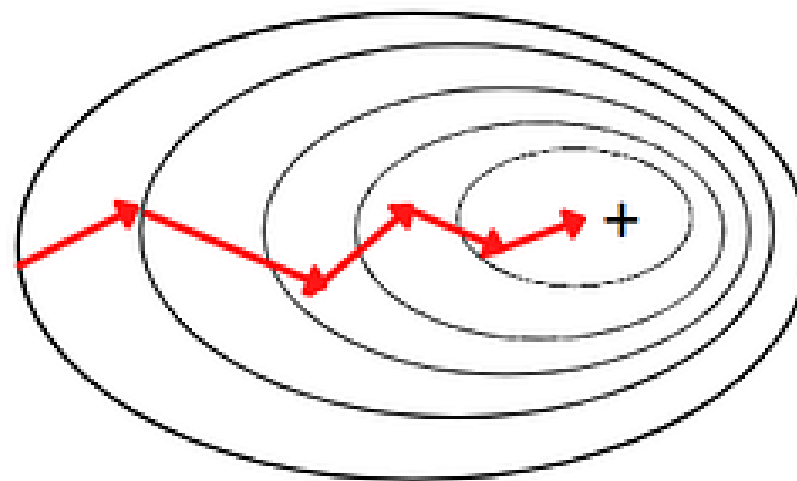




- ❑ Compromise between Batch and SGD
- ❑ GD computes gradient on a small batch of samples (e.g. 8, 16, 32, 64...)
- ❑ Most used technique in Deep Learning

- 👍 Faster convergence than Batch GD
- 👍 More stable results than SGD
- 👍 Efficient on GPU (vectorized computation)
- 👎 Still depends on batch size choice

Mini-Batch Gradient Descent



Optimizers

From vanilla Gradient Descent to optimizers

$$\theta_{t+1} = \theta_t - \underbrace{\eta \cdot \nabla L(\theta_t)}_{\text{Step size}}$$

New value points to θ_{t+1}
Old value points to θ_t
Gradient (vector) points to $\nabla L(\theta_t)$

θ : *vector of all parameters*

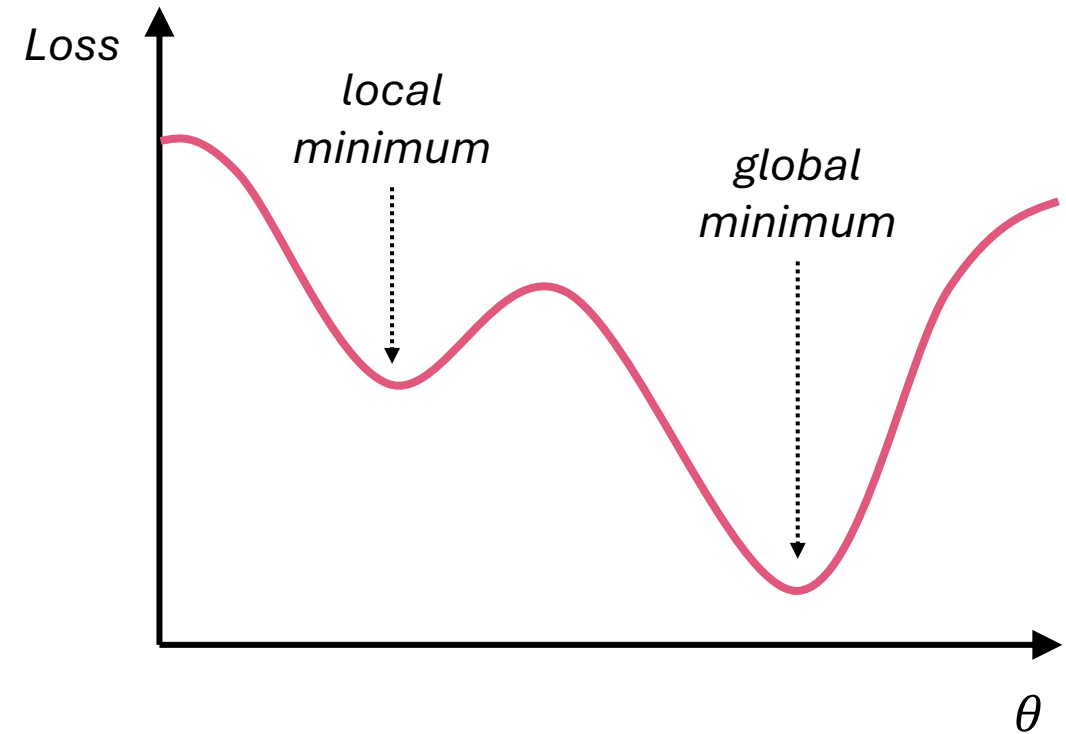
L : *loss function*

η : *learning rate*

- ❑ Vanilla GD defined the base rules to update model parameters at each training step
- ❑ This core idea remains the same but research has improved how the step is computed
- ❑ **Optimizers** are algorithms that adjust how parameters are updated during training
- ❑ Let's review the most important ones...

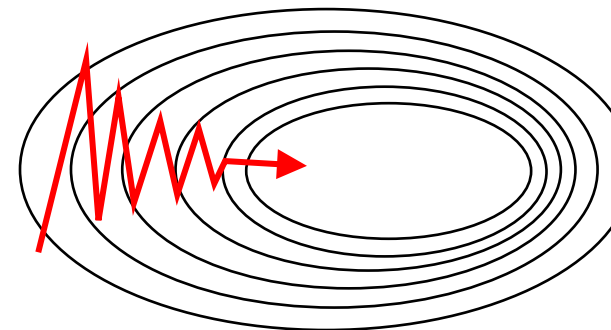
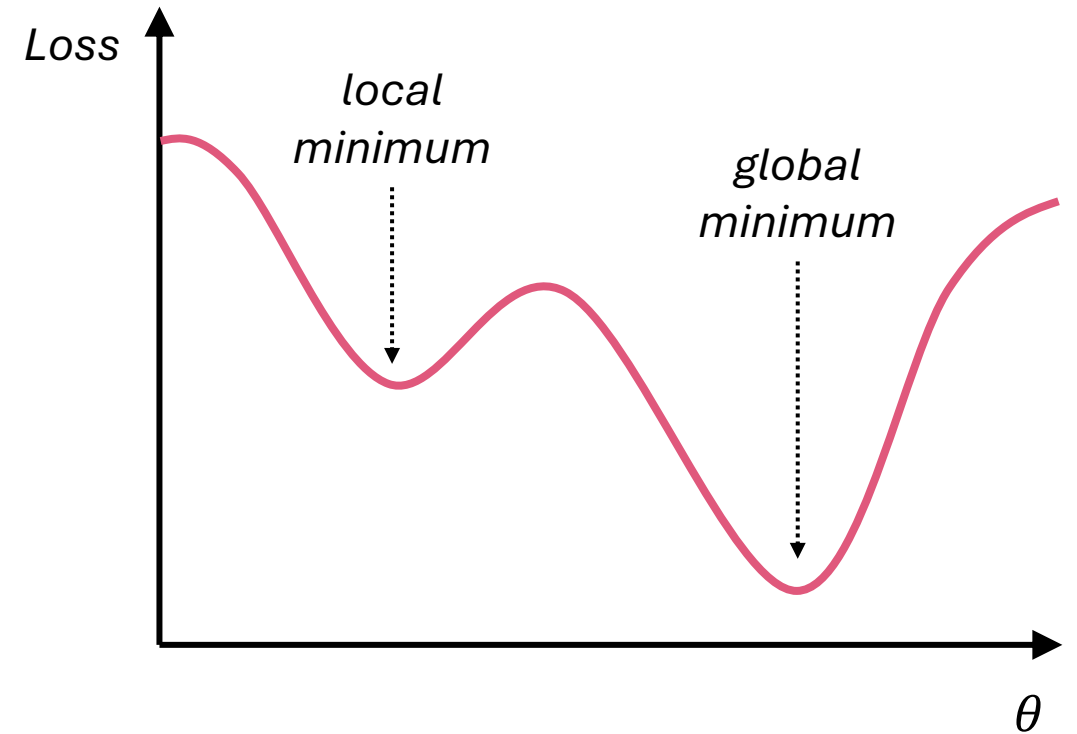
Gradient Descent with Momentum

- ❑ Momentum makes the update of the parameter not only based on the current gradient value but also on the previous ones
- ❑ It allow to addresses two issues with GD (consider stochastic or mini-batch)
- ❑ **Local minima**
 - A parameter optimized with a small learning rate could get stuck at a local minimum
 - Momentum adds some **inertia** to the descent, which takes into account **velocity** resulting from past iterations, helping to get out of the local minimum

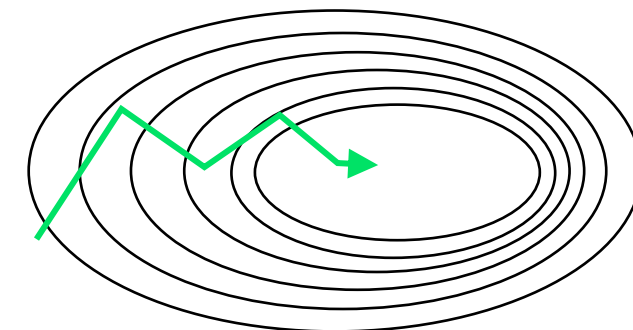


Gradient Descent with Momentum

- ❑ Momentum makes the update of the parameter not only based on the current gradient value but also on the previous ones
- ❑ It allow to addresses two issues with GD (consider stochastic or mini-batch)
- ❑ **Local minima**
 - A parameter optimized with a small learning rate could get stuck at a local minimum
 - Momentum adds some **inertia** to the descent, which takes into account **velocity** resulting from past iterations, helping to get out of the local minimum
- ❑ **Convergence speed**
 - Momentum smooths out oscillations and thus accelerates movement in consistent directions



Vanilla



+ Momentum

Gradient Descent with Momentum

- To keep track of previous gradients, a parameter $\beta \in [0, 1)$ controls the degree to which the gradient is smoothed over time

$$\mathbf{m}_t = \beta \cdot \mathbf{m}_{t-1} + (1 - \beta) \cdot \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \underbrace{\eta \cdot \mathbf{m}_t}_{\text{Step size}}$$

$\mathbf{m}$: *moment*

(exponentially weighted average of past gradients)

β : *momentum coefficient (e.g. 0.9)*

L : *loss function*

θ : *vector of parameters*

η : *learning rate*

AdaGrad (Adaptive Gradients)

- ❑ AdaGrad decreases learning rate over time
- ❑ Denominator sums up all the past gradients together, and never stops to increase
- ❑ Problem: learning rate shrinks too much over time

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\varepsilon + \sum_1^t (\nabla L(\theta_i))^2}}}_{\text{Step size}} \cdot \nabla L(\theta_t)$$

$\varepsilon : \approx 0$
 $L : \text{loss function}$
 $\theta : \text{vector of parameters}$
 $\eta : \text{learning rate}$

Gets smaller with time

RMS Prop (Root Mean Squared Propagation)



- ❑ RMS Prop is similar to AdaGrad, but uses exponentially weighted average of the squared gradients, giving more importance to recent gradients, less to older ones
- ❑ Learning rate's decrease is thus more adaptative

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot (\nabla L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\epsilon + v_t}}}_{\text{Step size}} \cdot \nabla L(\theta_t)$$

v : Squared – gradient moment
(exponentially weighted average of squared past gradients)

β : coefficient (e.g. 0.9)

L : loss function

θ : vector of parameters

η : learning rate

Gets smaller
with time



- ❑ Adam is very efficient and has become **the default optimizer in deep learning**
- ❑ It combines both Momentum + RMS Prop concepts + adds a correction to $\mathbf{m}_t$ and $\mathbf{v}_t$
- ❑ Recent observations will always have more importance

$$\theta_{t+1} = \theta_t - \underbrace{\frac{\eta}{\sqrt{\epsilon + \hat{v}_t}}}_{\text{step size}} \cdot \hat{\mathbf{m}}_t$$

$\hat{\mathbf{m}}$: biases corrected gradients moment

$\hat{v}$: biases corrected squared gradients moment

L : loss function

θ : vector of parameters

η : learning rate



□ Why moments m_t and v_t are biased?

- Since both start at 0, the first values of m_t and v_t are too small (biased towards 0)
- They do not provide a good estimate of the first moving average!

$$\begin{array}{ccc} m_t = \beta \cdot m_{t-1} + (1 - \beta) \cdot g_t & \xrightarrow{\text{let } \beta = 0.9} & m_1 = 0.1 g_1 \\ v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2 & & v_1 = 0.1 g_1^2 \end{array}$$



- Why moments m_t and v_t are biased?
 - Adam's bias-corrected estimates fix this underestimation with a correction β^t
 - As t grows, $\beta^t \rightarrow 0$ So, correction becomes negligible

$$\begin{array}{ccc} \widehat{m}_t = \frac{m_t}{(1 - \beta^t)} & \xrightarrow{\text{let } \beta = 0.9} & \widehat{m}_1 = \frac{0.1 \cdot g_1}{0.1} \\ \widehat{v}_t = \frac{v_t}{(1 - \beta^t)} & & \widehat{v}_1 = \frac{0.1 \cdot g_1^2}{0.1} \end{array}$$